UML与面向对象建模



内容提要

1. UML发展历史
2. UML的基本概念
3. 静态建模
4. 动态建模
5. 物理架构
6. 建模步骤



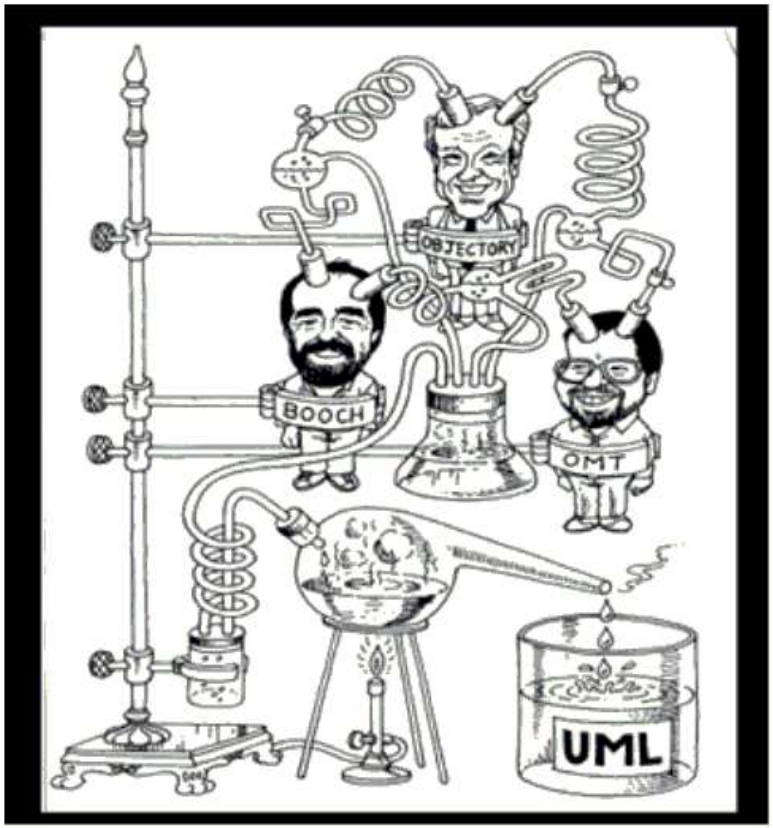
统一建模语言UML

1 发展历史

UML的产生

1988年到1992年是OO方法学蓬勃发展的时期，人们从各自的经历和软件开发的经验提出了各种OO的开发方法，代表的有：

- Grady Booch的Booch 方法
- James Rumbaugh的对象建模技术(OMT)
- Ivar Jacobson的面向对象软件工程(OOSE)

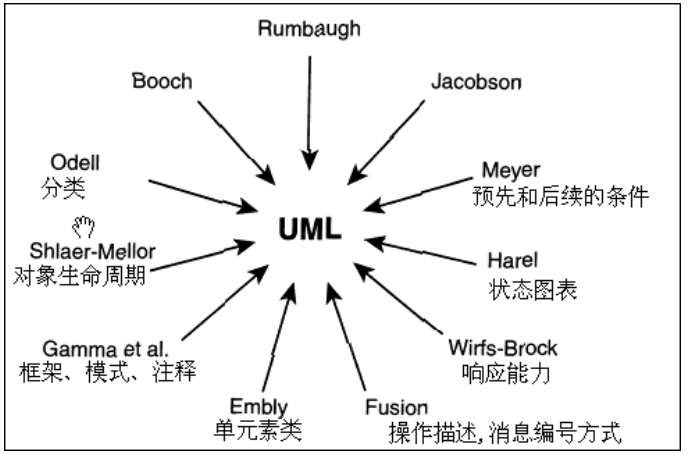


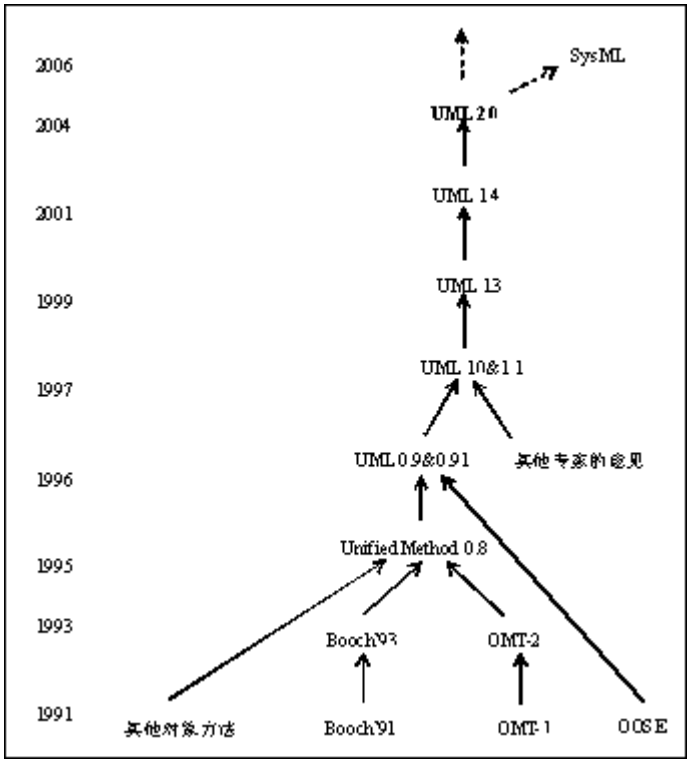
他们是 RUP 和 UML 的共同创始人，被誉为 RUP 和 UML 之父，并被人们冠以“the Three Amigos”的美誉。

http://www.ibm.com/developerworks/cn/rational/theme/rational-rup/

（续）

- 1994年Rational公司的Grady Booch与新加盟的James Rumbaugh 联合，开始了统一OO方法学和工具的研发历程。以融合Booch和OMT方法的UML开发开始。
- 1995年10月UML0.8发布。
- 1995年秋，Ivar Jacobson和他的 Objectory 公司加盟Rational，UML中加入了OOSE方法，使其包容了当时最实用的各种OO方法。
- 1996年，UML0.9版本发布，1997年1月，UML1.0被提交给OMG组织，作为软件建模语言的候选。
- 1997年11月7日，UML1.1正式被OMG组织采纳为业界标准。UML经历了1.2、1.3、1.4。
- UML2.0版本于2003年6月份通过。





Formal Versions

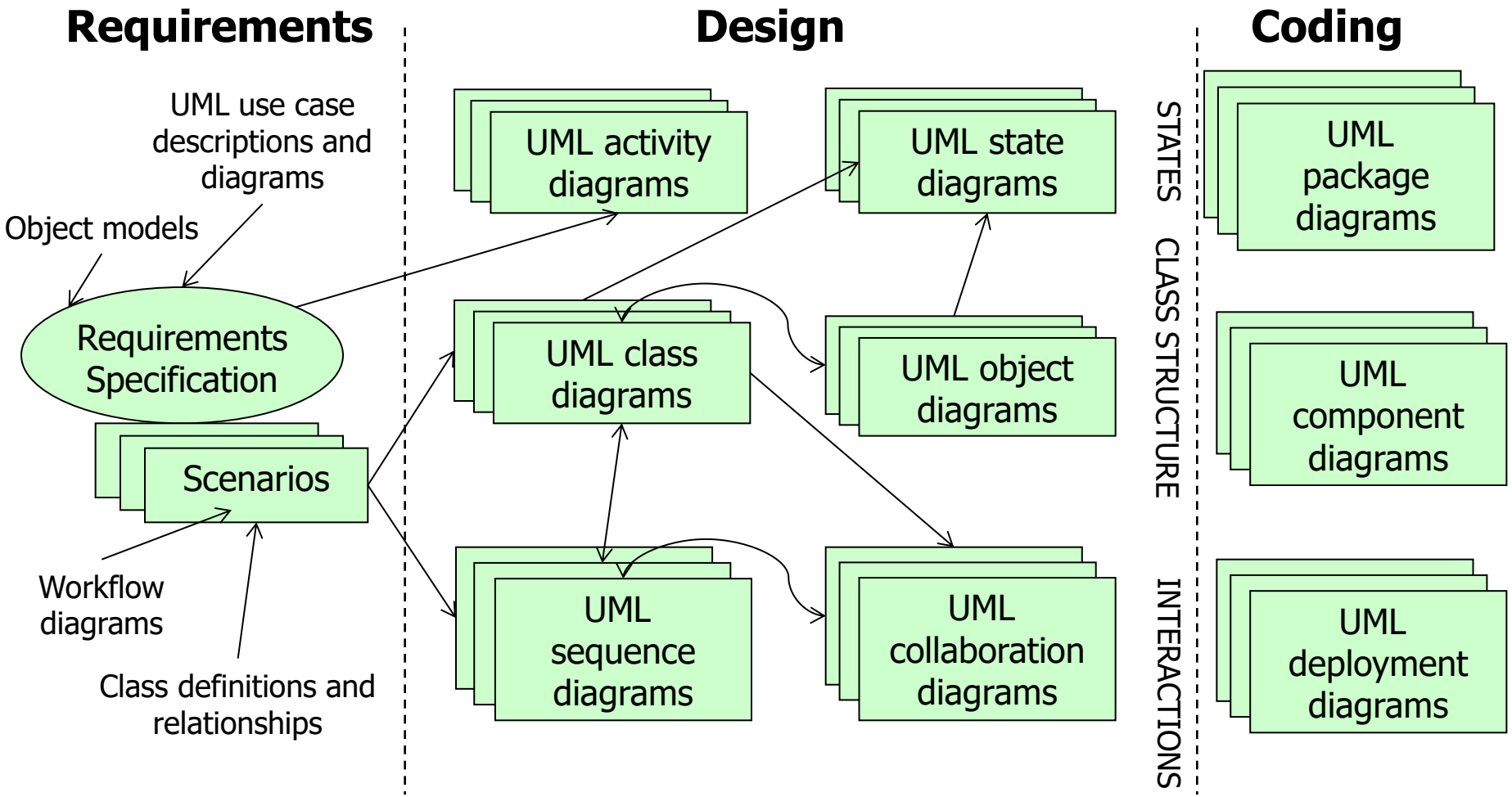
https://www.omg.org/spec/UML

VERSION	ADOPTION DATE	URL
2.5.1	December 2017	https://www.omg.org/spec/UML/2.5.1
2.4.1	July 2011	https://www.omg.org/spec/UML/2.4.1
2.3	May 2010	https://www.omg.org/spec/UML/2.3
2.2	January 2009	https://www.omg.org/spec/UML/2.2
2.1.2	October 2007	https://www.omg.org/spec/UML/2.1.2
2.0	July 2005	https://www.omg.org/spec/UML/2.0
1.5	March 2003	https://www.omg.org/spec/UML/1.5
1.4	September 2001	https://www.omg.org/spec/UML/1.4
1.3	February 2000	https://www.omg.org/spec/UML/1.3
1.2	July 1999	https://www.omg.org/spec/UML/1.2
1.1	December 1997	https://www.omg.org/spec/UML/1.1

UML的应用

- UML的主要目标是以面向对象图的方式来描述任何类型的系统。最常用于建立软件系统的模型，也可描述非软件领域的系统，如：机械系统、企业机构、业务过程、信息系统、 实时的工业系统和工业过程等。
- “以图规划”和“按图施工”。
- UML是一种建模语言，是一种标准的表示方法，而不是一种完整的方法学。UML的图是应用的基础——**为不同领域的人提供统一的交流方法**。
- UML适用于系统开发的全过程，它的应用贯穿于从需求分析到系统建成后测试的各个阶段。

How UML supports the development process



（续）

- 需求分析：可以用用例来捕获用户的需求。通过用例建模，可以描述对系统感兴趣的外部角色及其对系统的功能要求（用例）。
- 分析：分析阶段主要关心问题域中的类（对象），需要识别这些类以及它们相互间的关系：
  - 可以用UML的逻辑视图和动态视图来描述
  - 类图描述系统的静态结构
  - 协作图、顺序图、活动图和状态图描述系统的动态行为。
- 设计：把分析阶段的结果扩展成技术解决方案，加入新的类来定义软件系统的技术方案细节。设计阶段用和分析阶段类似的方式使用UML。

（续）

- 构造(编码)：这个阶段的任务是把来自设计阶段的类转换成某种面向对象程序设计语言的代码。
- 测试：UML模型可作为测试阶段的依据，不同测试小组使用不同的UML图作为他们工作的依据：
  - 单元测试使用类图和类规格说明；
  - 集成测试使用构件图和协作图；
  - 系统测试使用用例图来验证系统的行为；
  - 验收测试由用户进行，用与系统测试类似的方法，验证系统是否满足在分析阶段确定的所有需求。

对UML的认识

- UML的初衷是为了结束OO领域的方法混战。
- UML是一个通用的、标准的建模语言，对任何有静态结构、 动态行为的系统都可用来建模。
- 方法与建模语言不同，UML不是标准的开发过程， 也不是标准的面向对象开发方法。
- 不能仅仅依靠UML编写可运行的程序，UML是一套符号系统，仅仅是一种简化和辅助沟通的工具。
- 已成为国际软件界广泛承认的标准，它是第三代面向对象的开发方法，是一种基于面向对象的可视化的通用建模语言。

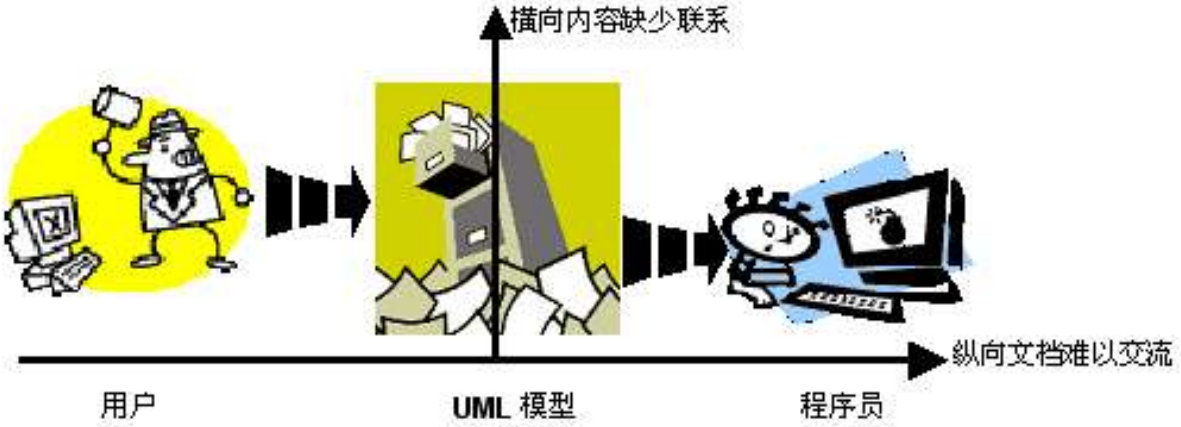
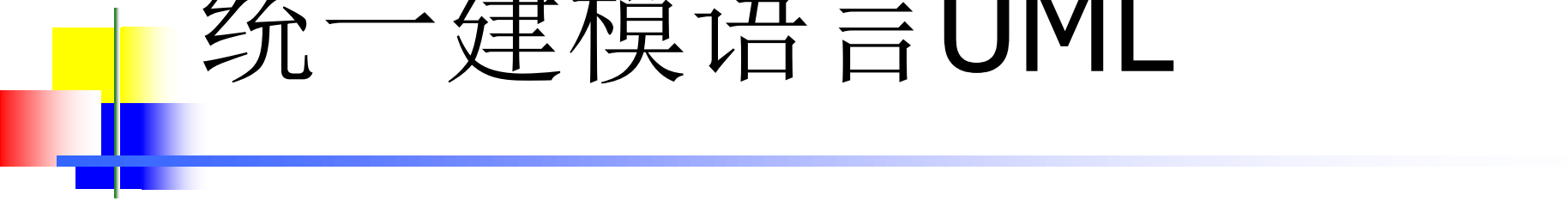


图 采用UML描述的建模结果“分崩离析”



统一建模语言UML

2 UML的基本概念

UML的基本概念

- UML的语义
- UML表示法



UML的语义

- 元模型为UML的所有元素在语法和语义上提供了简单、一致及通用的定义性说明。 使开发者能在语义上取得一致，消除因人而异的表达方法。
- UML语义还支持对元模型的扩展定义。
- UML的元素是基本构造单位，其中模型元素用于构造系统，视图元素用于构成系统的表示部分。
- UML定义的各种机制的语义，保持了UML的简单和概念上的一致。这些机制是依赖关系、约束、注记、标记值和定制等。
- UML支持的各种类型的语义，如布尔、表达式、列表、 值、名字、坐标、字符串和时间等，还允许用户自定义类型。

UML表示法

- UML的图形符号，为建模者和建模工具的开发者提供了标准的图形符号和正文语法。
- UML表示法的构成：
  - 视图（view）
  - 图（diagram）
  - 模型元素（model element）
  - 通用机制（general mechanism）
- UML表示法分为两部分：
  - 通用表示
  - 图形表示



通用表示

① 字符串：用于表示有关模型的信息。
② 名字：用于表示模型元素。
③ 标号：用于表示缚于图形符号的字符号。
④ 特定字符串：用于表示缚于模型元素的特性。
⑤ 类型表达式：用于声明属性变量和参数。
⑥ 定制：是一种机制，用已有的模型元素来定义新的模型元素。

图形

① 用例图（use-case diagram）
- 表示各类外部参与者与系统所提供的用例之间的连接。
- 一个用例是系统所提供的一个功能的描述。
- 给出了用户所感受到的系统行为，但不描述系统如何实现该功能。

② 类图（class diagram）
- 表示系统中类的静态结构，即类与类之间的相互联系。
- 一个系统可以有多张类图，一个类也可以出现在几张类图中。

③ 对象图（object diagram）
- 对象图是类的实例，它表示系统执行在某一时间点的一个可能的快照（snapshot）。

④ 状态图（state diagram）
- 状态图通常是对类描述的补充，它说明该类的对象所有可能的状态以及哪些事件将导致的改变。

（续）

⑤ 时序图（ sequence diagram）
- 时序图表示几个对象之间的动态协作关系。
- 它主要用来分析对象之间发送消息的顺序。

⑥ 协作图（collaboration/communication diagram）
- 与时序图一样，协作图表示对象间的动态协作关系。
- 它除了说明消息的交互外，还显示对象及其间的关系。

⑦ 活动图（activity diagram）
- 表示连续的活动流，通常用来描述一个操作所需要的活动。

⑧ 构件图（component diagram）
- 以代码构件为单位表示代码的物理结构，及构件之间的依赖关系。

⑨ 部署图（deployment diagram）
- 表示系统中硬件和软件的物理结构，及各部分之间的连接方式。

视图

① 用例视图（use-case view）
- 表示外部参与者看到的系统将提交的功能。

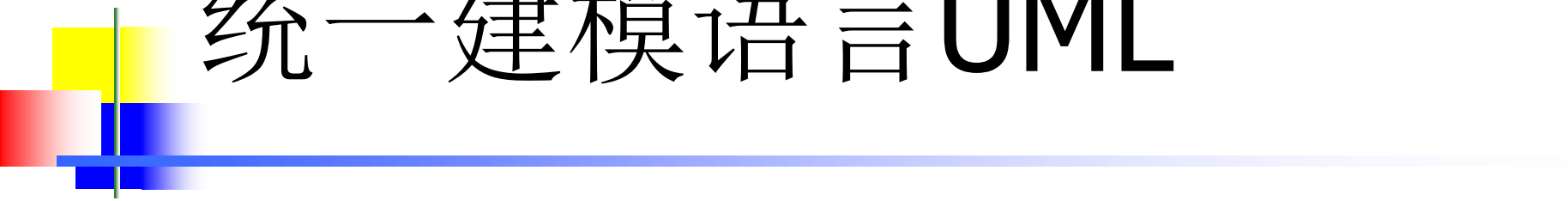
② 逻辑视图（logical view）
- 表示系统内部如何提供系统的功能。
- 静态结构用类图和对象图来描述，动态行为用状态图、时序图、协作图、活动图来描述。

③ 构件视图（component view）
- 构件视图表示代码构件的组织，描述了实现模块及它们之间的依赖关系。
- 构件视图针对开发者。

（续）

③ 进程视图（process view）
- 表示系统的并发性，包括进程的通信和同步。
- 用动态图（即状态图、时序图、协作图和活动图）和实现图（即构件图和部署图）来描述。
- 针对开发者和系统集成者。

⑤ 部署视图（deployment view）
- 表示系统的物理部置。
- 构件被部署到物理结构中的映射，用部署图来描述。
- 它针对开发者、集成者和测试者。



统一建模语言UML

3 UML的主要内容

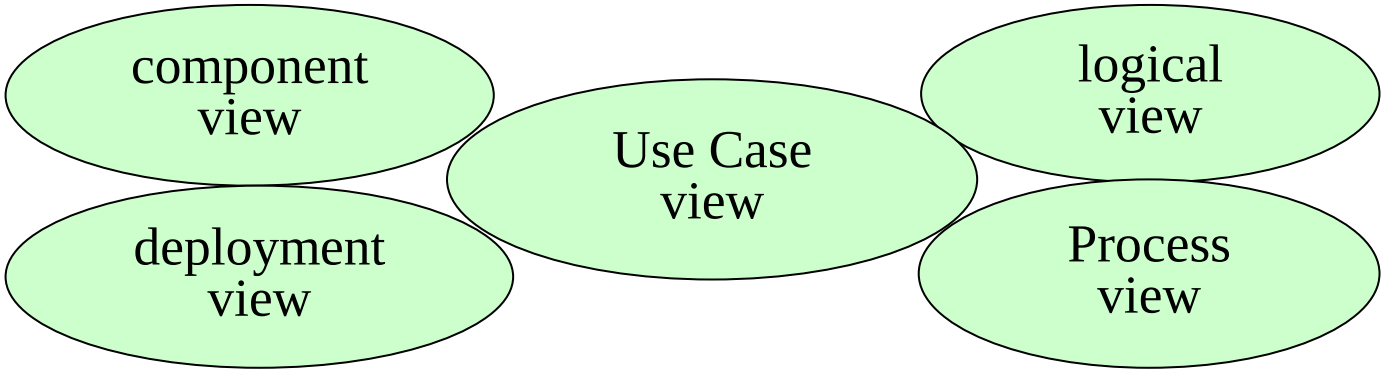
UML的主要内容

- UML视图
- UML基本元素



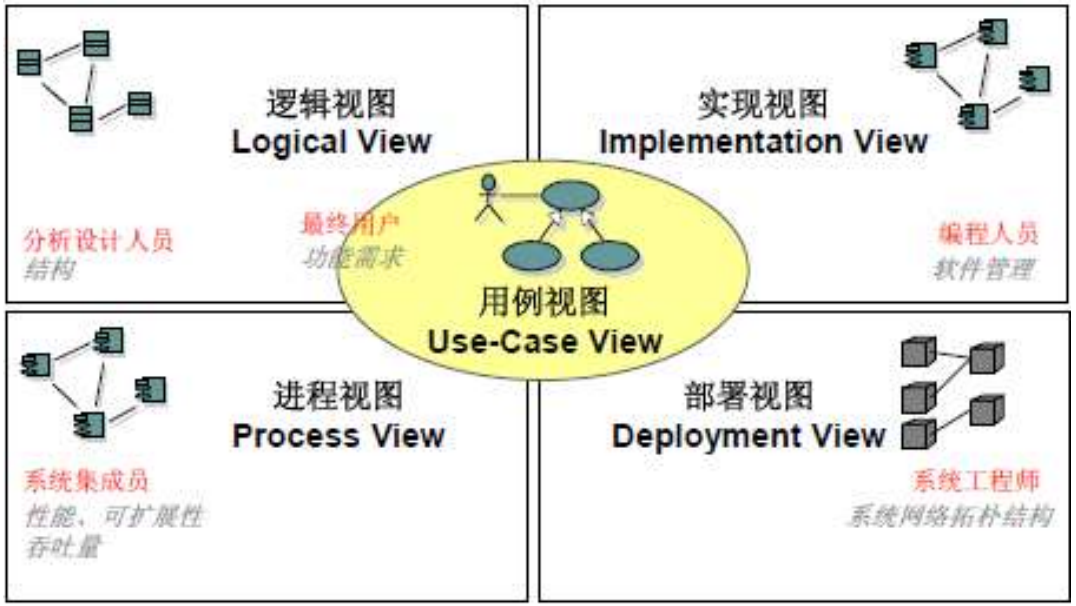
视图

- 一个系统应从不同的角度进行描述，从一个角度观察到的系统称为一个视图（view）。
- 视图由多个图（diagram）构成，它不是一个图表（graph），而是在某一个抽象层上，对系统的抽象表示。
- 如果要为系统建立一个完整的模型图，需定义一定数量的视图，每个视图表示系统的一个特殊的方面。另外，视图还把建模语言和系统开发时选择的方法或过程连接起来。

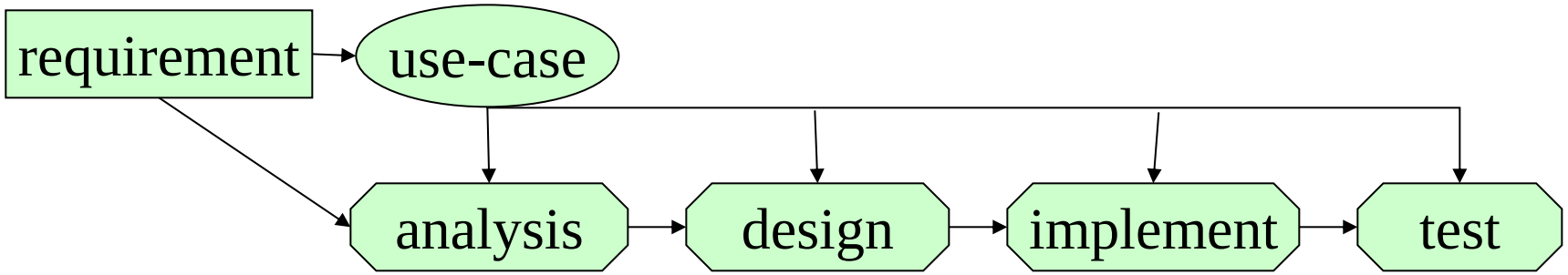


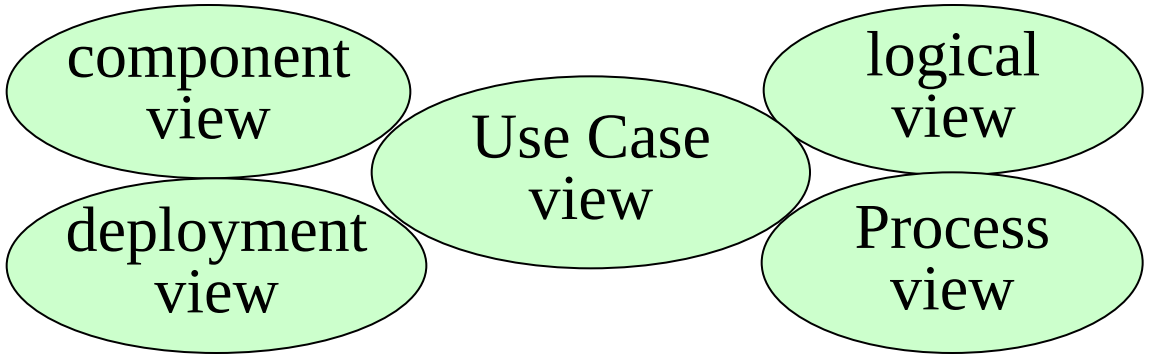
系统描述与视图

- UML描述系统的方式：
  - 系统需求
  - 静态结构
  - 动态行为
  - 交互行为
  - 物理实现



Use-case affect all views and phrases



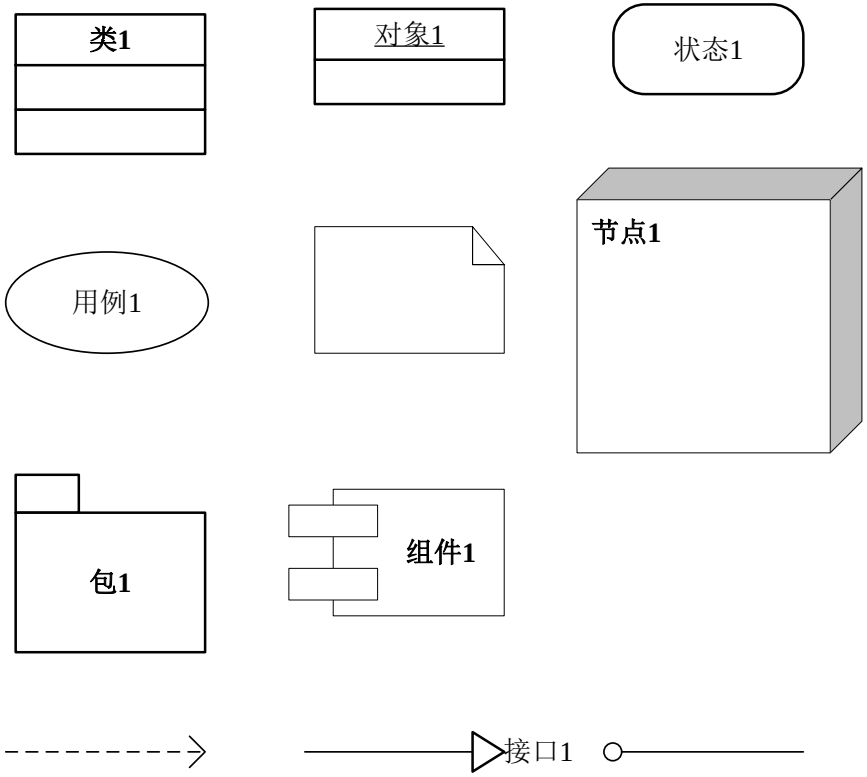


基本元素

- 分类：
  - 参与者（ator）
  - 用例（use case）
  - 类（class）
  - 对象（object）
  - 接口（interface）
  - 子系统（subsystem）
  - 包（package）
  - 组件（component）
  - 结点（node）
  - 注释（comment)
- 关系：
  - 关联（association）
  - 泛化（generalization）
  - 依赖（dependency）
  - 实现（realization）
  - 约束（constraint)

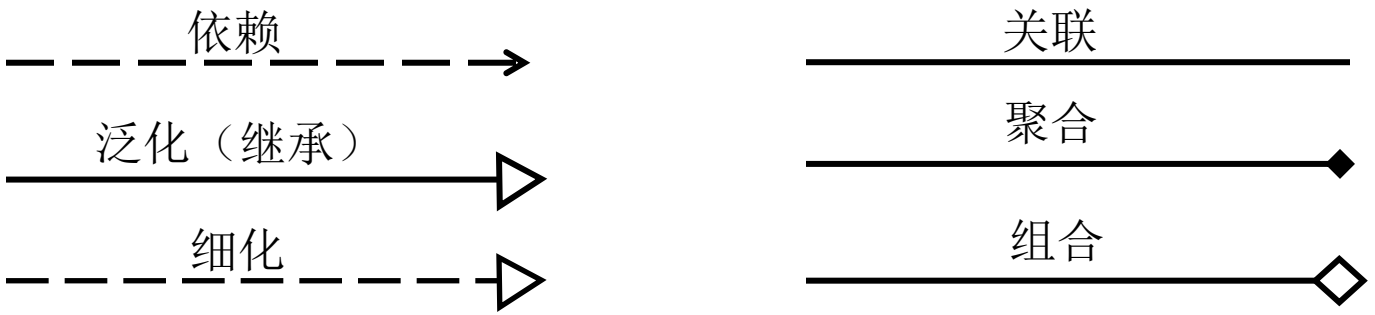


模型元素



基本元素—关系

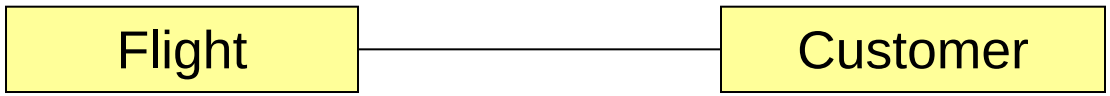
模型元素与模型元素之间的连接关系也是模型元素，常见的关系有关联（association）、泛化（generalization）、依赖（dependency）和聚合（aggregation），其中聚合是关联的一种特殊形式。



- 关联：连接（connect）模型元素及链接（link）实例。
- 依赖：表示一个元素以某种方式依赖于另一种元素。
- 泛化：表示一般与特殊的关系，即“一般”元素是“特殊”关系的泛化。
- 聚合：表示整体与部分的关系。

基本元素—关系—关联

- 关联（association）是类之间的语义连接（link），使一个类知道另一个类的公共属性和操作。
- 关联可以是单向的也可以是双向的。

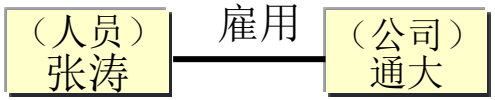


关联的表示

- 关联有二元关联(binary)、三元关联(ternary)、多元关联(higher order)。

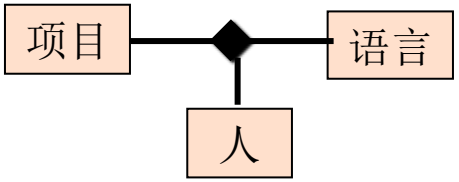


二元关联的例

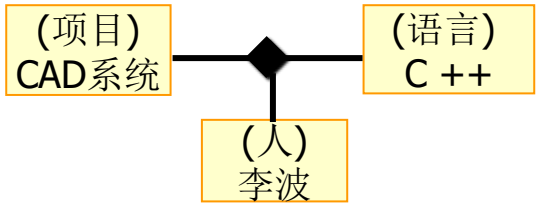


链的例子

（a） 二元关联



三元关联的例

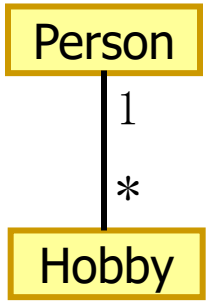


链的例子

（b）三元关联

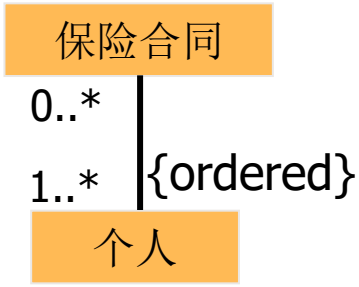
关联的重数

- 重数（multiplicity）表示多少个对象与对方对象相连接，它的默认值为1，常用的重数表示符号有：
  - 0..1　　表示零或1
  - 0..* 或*　　表示零或多个
  - 1..*　　表示1或多个
  - 1,3,7　　表示1、3或7（枚举型）

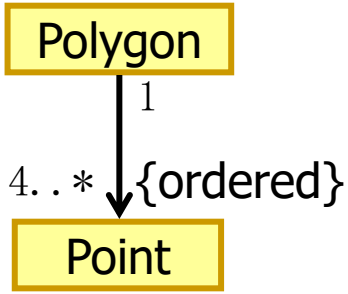


带有多重性关联

- 在关联的多端标注{ordered}指明这些对象是有序的。
- 关联可以用箭头，表示该关联使用的方向（单向或双向），称为导引或导航（navigation）。



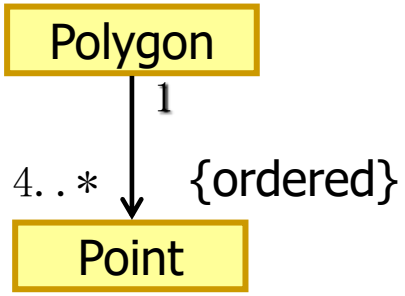
(a)指定链接之间有明确的顺序

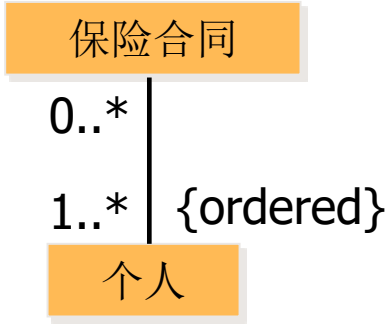


(b)单向关联

关联的约束

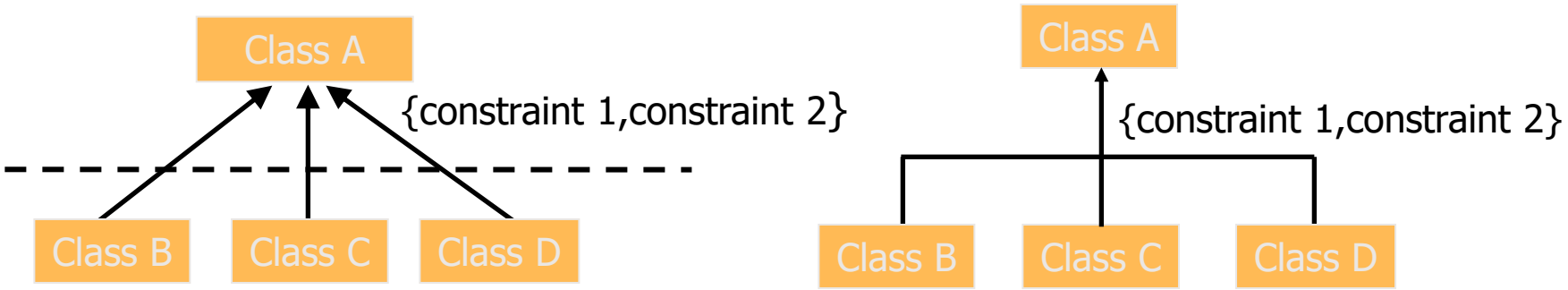
- 约束（constraint）是各种模型元素的一种语义条件或限制。一条约束只能应用于同一类的元素。
- 如果约束应用于一种具有相应视图元素的模型元素，它可以出现在它所约束元素视图元素的旁边。
- 通常一个约束由一对花括号括起来（{constraint}），花括号中为约束内容。
- 如果一条约束涉及同一种类的多个元素，则要用虚线把所有受约束的元素框起来，并把该约束显示在旁边（如或约束）。





泛化的约束

- 约束可分为：泛化的约束、关联的约束。
- 对泛化的约束
  - 显示在大括号里，若有多个约束，用逗号隔开。
  - 如果没有共享，则用一条虚线通过所有继承线，并在虚线的旁边显示约束。



对泛化的约束的两种表示方法

对泛化有以下常用的约束：

① complete：泛化中所有子元素都已在模型中说明，不允许再增加其它子元素。

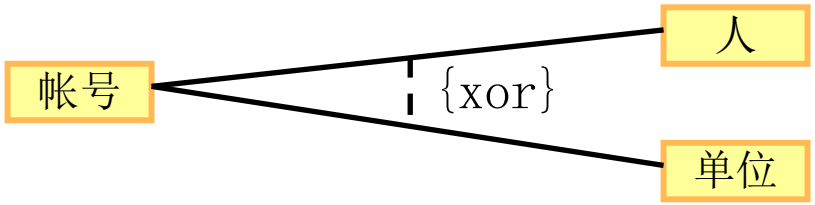
② incomplete：不是泛化中所有子元素都已说明，允许再增加其它子元素。

③ disjoint：父类对象不能有多于一个型的子对象。

④ overlapping：给定父类对象可有多于一个型的子对象，表示重载。

关联的约束

- 对关联有以下常用的约束：
  ① implicit：该关联只是概念性的，在对模型进行精化时不再用。
  ② ordered：具有多重性的关联一端的对象是有序的。
  ③ changeable：关联对象之间的链（link）是可变的（添加、修改、删除）。
  ④ addonly：可在任意时刻增加新的链接。
  ⑤ frozen：冻结已创建的对象，不能再添加、删除和修改它的链接。
  ⑥ xor： “或约束”，某时刻只有一个当前的关联实例。

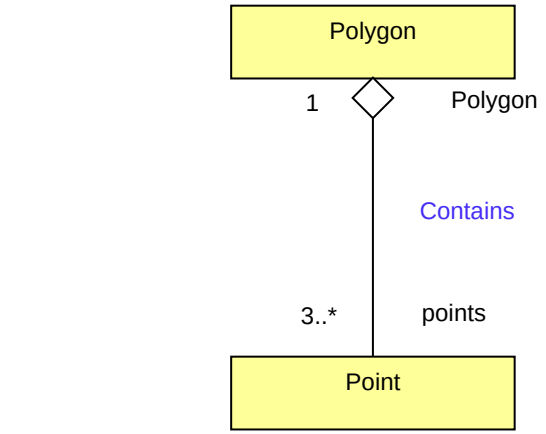


对象类的xor关联

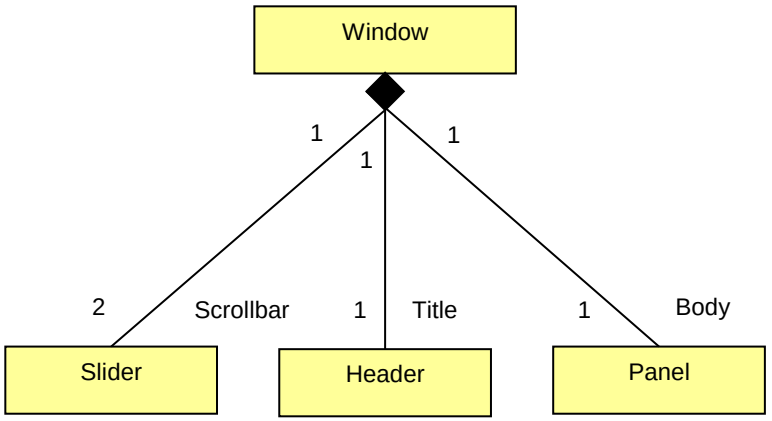
- 对消息，链接角色和对象的约束
- 自定义约束

基本元素—关系—聚集

- 聚集（Aggregation）用来表达整体——部分关系的关联。
- 聚集是特殊的关联，它可以有重数、角色、限制等。聚集分为：
  - ◇ 共享（shared）聚集，它的“部分”对象可以是任意“整体”对象的一部分。
  - ◆ 组合（Composition）聚集，是指它的“整体”拥有它的“部分”，是关联更强的形式。



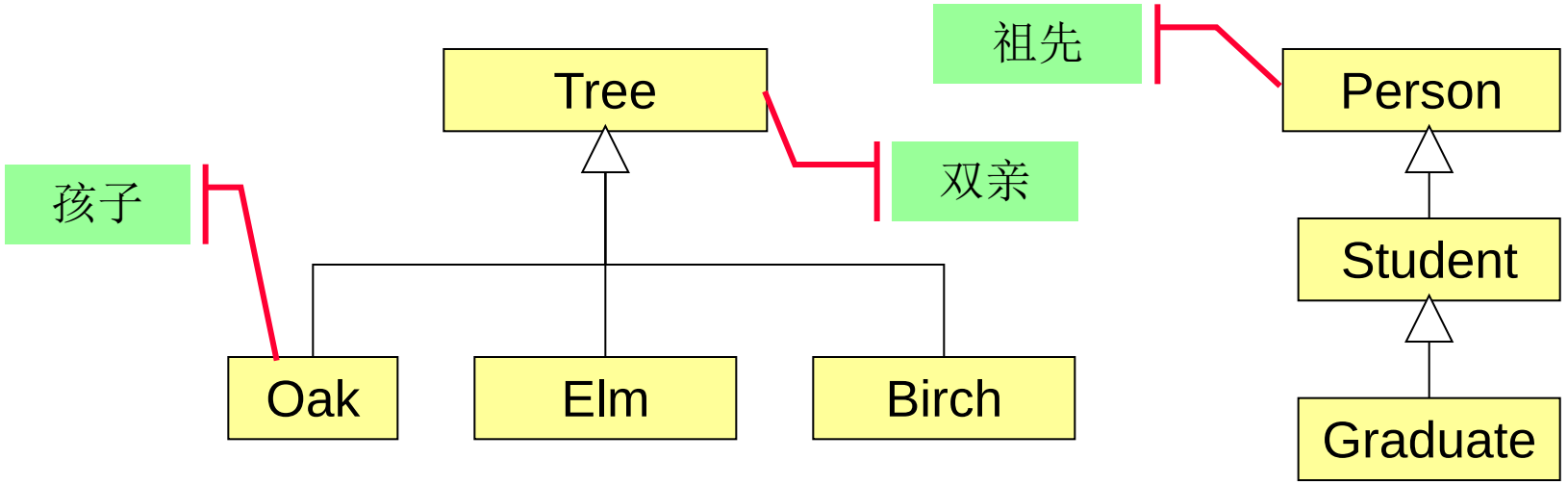
共享聚集



组合聚集

基本元素—关系—泛化

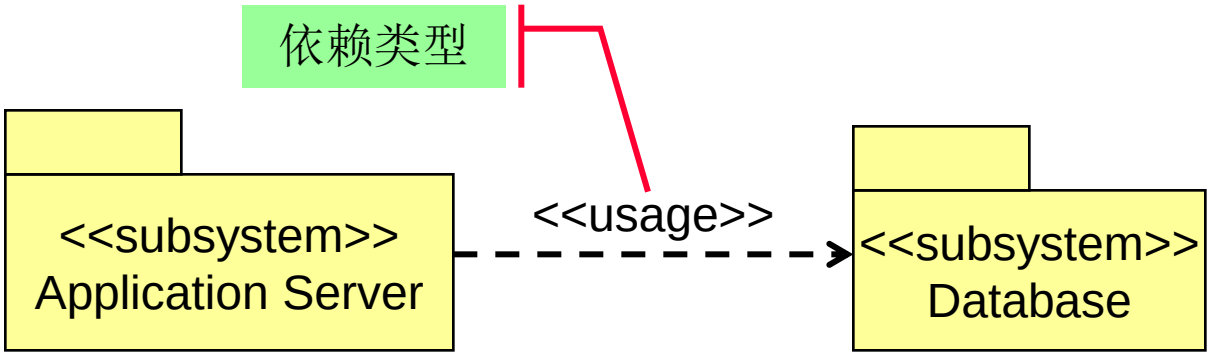
- 泛化（generalization）是一般和具体之间的一种关系。
- 继承（inherit）就是一种泛化关系，更一般化的描述称为双亲，双亲的双亲称为祖先，更具体化的描述称为孩子，在类的范畴，双亲对应超类，孩子对应派生类。



基本元素—关系—依赖

- 依赖（dependency）指明一个元素引用另一个元素的关系，它可分为类与类之间的依赖和包与包之间的依赖。
- 依赖有很多种类，比如：

实现（realize）	使用（usage）	实例化（instantiate）
调用（call）	派生（derive）	访问（access）
引入（import）	友元（friend）	……



依赖的形式

依赖的形式可能是多样的，针对不同的依赖的形式，依赖关系有不同的变体(varieties)：

① 抽象(abstraction)：从一个对象中提取一些特性，并用类方法表示。

② 绑定(binding)：为模板参数指定值，以定义一个新的模板元素。

③ 组合(combination)：对不同类或包进行性质相似融合。

④ 许可(permission)：允许另一个对象对本对象的访问。

⑤ 使用(usage)：声明使用一个模型元素需要用到已存在的另一个模型元素，这样才能正确实现使用者的功能（包括调用、实例化、参数、发送）。

⑥ 跟踪(trace)：声明不同模型中元素的之间的存在一些连接。

⑦ 访问或连接(access)：允许一个包访问另一个包的内容。

（续）

⑧ 调用(call)：声明一个类调用其他类的操作的方法。

⑨ 导出(derive)：声明一个实例可从另一个实例导出。

⑩ 友元(friend)：允许一个元素访问另一个元素，不管被访问的元素是否具有可见性。

11 引入(import)：允许一个包访问另一个包的内容，并为被访问组成部分增加别名。

12 实例(instantiation)：关于一个类的方法创建了另一个类的实例声明。

13 参数(parameter)：一个操作和它参数之间的关系。

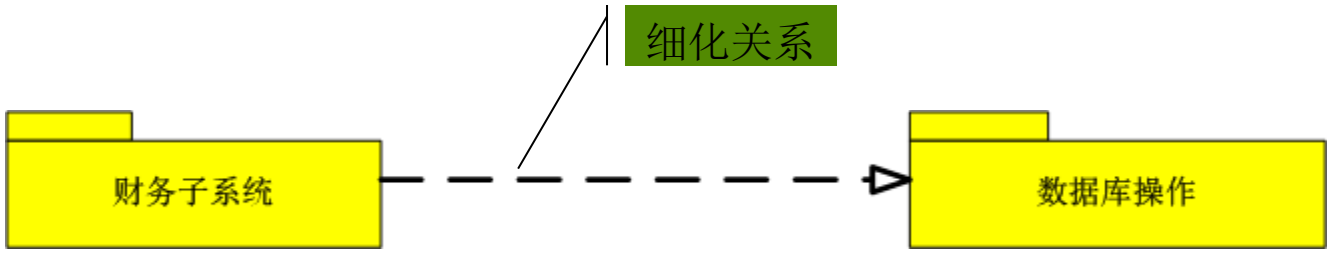
14 实现(realize)：说明和其实之间的关系。

15 精化(refine)：声明具有两个不同语义层次上的元素之间的映射。

16 发送(send)：信号发送者和信号接收者之间的关系。

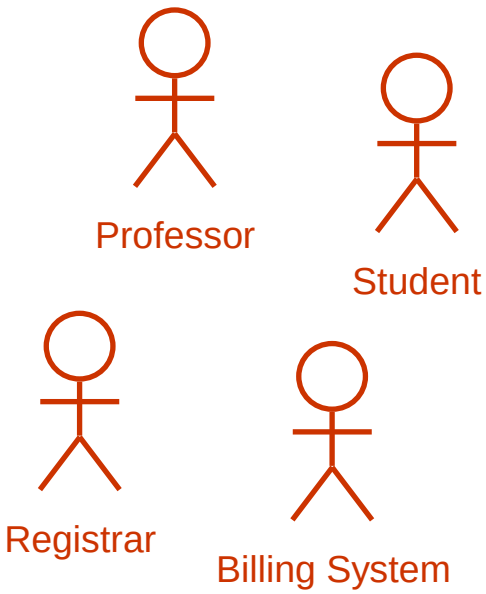
基本元素—关系—细化

- 细化（refinement）是对同一事物不同级别的两种描述之间的一种关系。
- 例如：对同一事物建模的分析类与设计类之间的就是细化关系。



基本元素—参与者（actor）

- 参与者是与系统、子系统或类交互的外部人员，进程或事务。在运行时，具体人员会充当系统的多个参与者，不同用户可能会成为一个参与者。

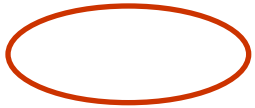


- **如何确定参与者：**

① 谁使用系统的主要功能(主参与者)?
② 谁需要从系统获得对日常工作的支持和服务?
③ 需要谁维护管理系统的日常运行（副参与者）?
④ 系统需要控制哪些硬件设备?
⑤ 系统需要与其它哪些系统交互?
⑥ 谁需要使用系统产生的结果（值）?

基本元素—用例（use case）

- 用例是系统提供的外部可感知的功能单元，它的作用是定义清晰的系统行为，但不解释系统的内部结构。



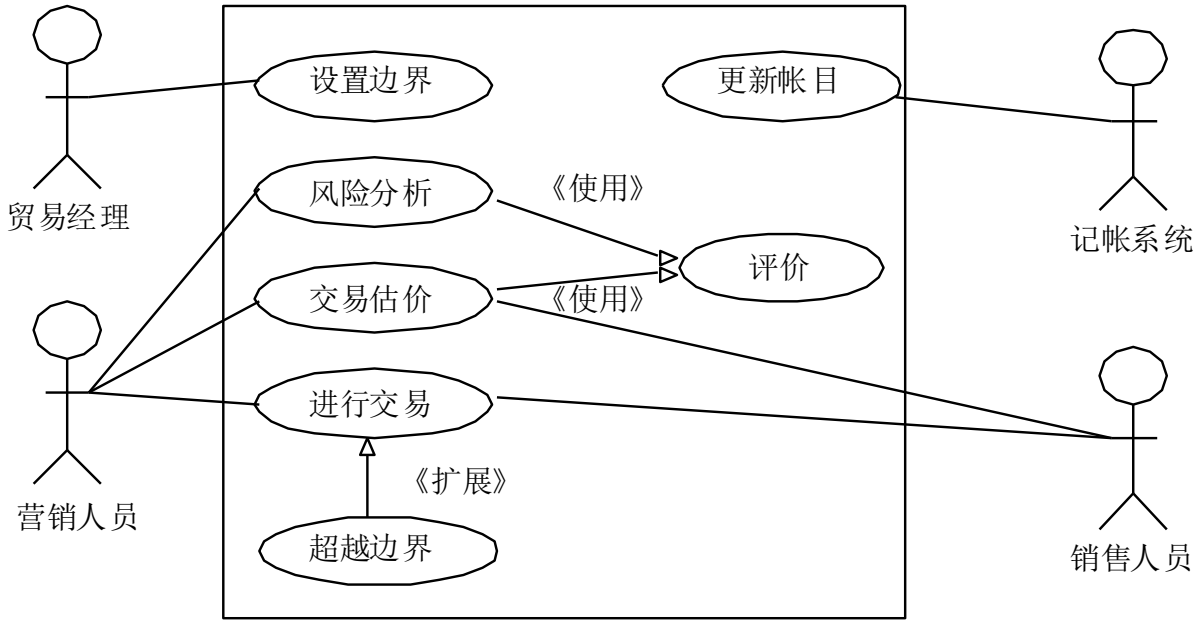
Maintain Curriculum

用例用椭圆来表示，用例名标在椭圆下方，用实线与同自身通信的用户相连。

- **如何确定用例：**

①与系统实现有关的主要问题是什么？
②系统需要哪些输入/输出？这些输入/输出从何而来？到哪里去？
③参与者需要系统提供哪些功能？
④参与者是否需要对系统中的信息进行读、创建、修改、删除或存储？

例：金融贸易系统的用例图



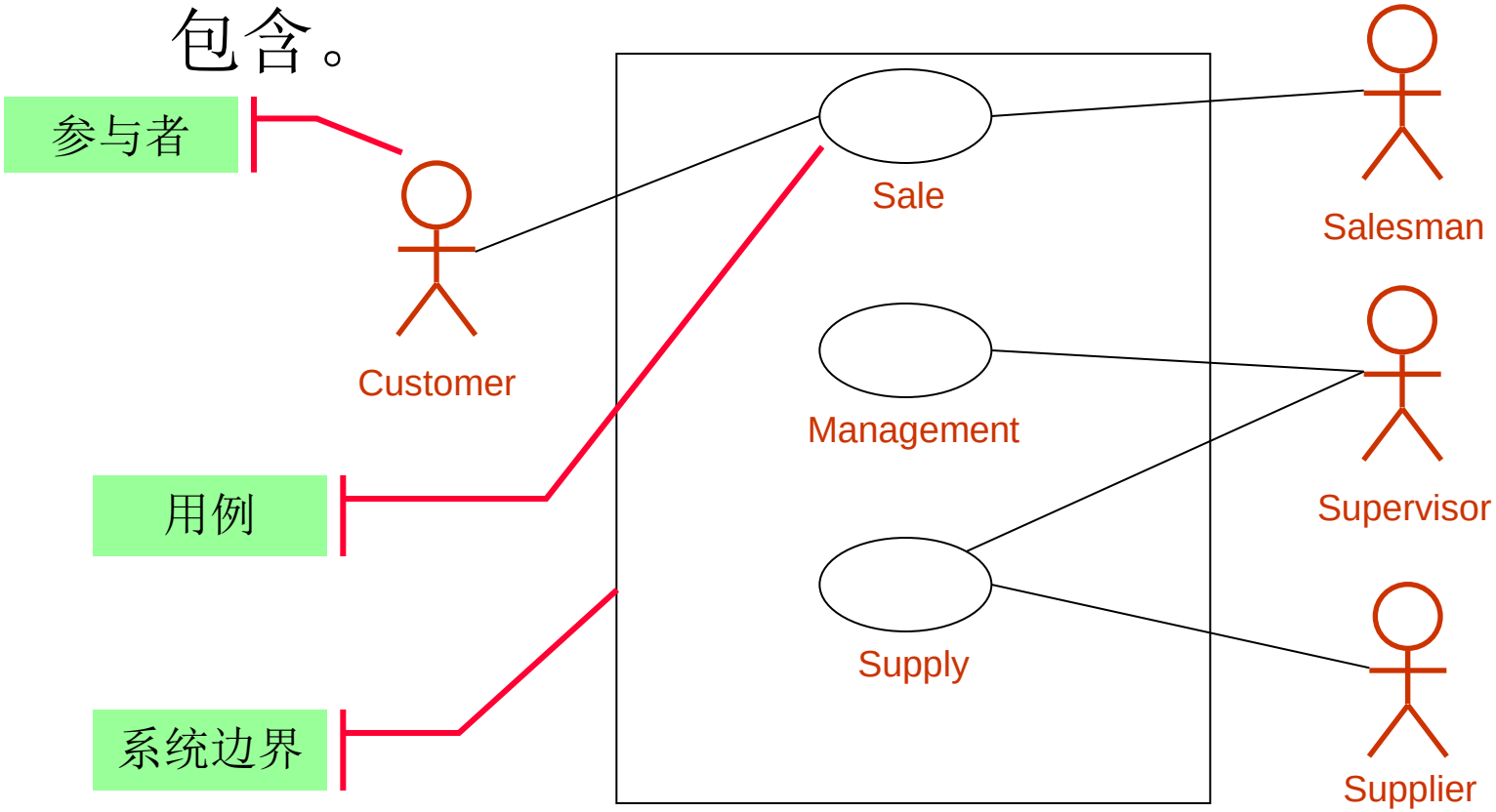
- 用例可以与参与者关联，也可以参与其他多种关系，比如扩展、泛化和包含等。
- 用例有以下特点：
  - 用例捕获某些用户可见的需求，实现一个具体的用户目标。
  - 用例由参与者激活，并将结果值反馈给参与者。
  - 用例必须具有功能上的完整描述。

用例图（use-case diagram）

- 用例图描述各个参与者在各个用例中的参与情况，描述系统为用户所感知的外部视图。
- 用例图的功能：
  - 捕获用户需求
  - 描述系统边界
  - 指明系统外部行为
  - 指导系统开发者的功能开发
  - 系统建模的起点，指导所有的类图和交互图的设计
  - 产生测试用例，用户文档
  - 估计项目大小和进度。

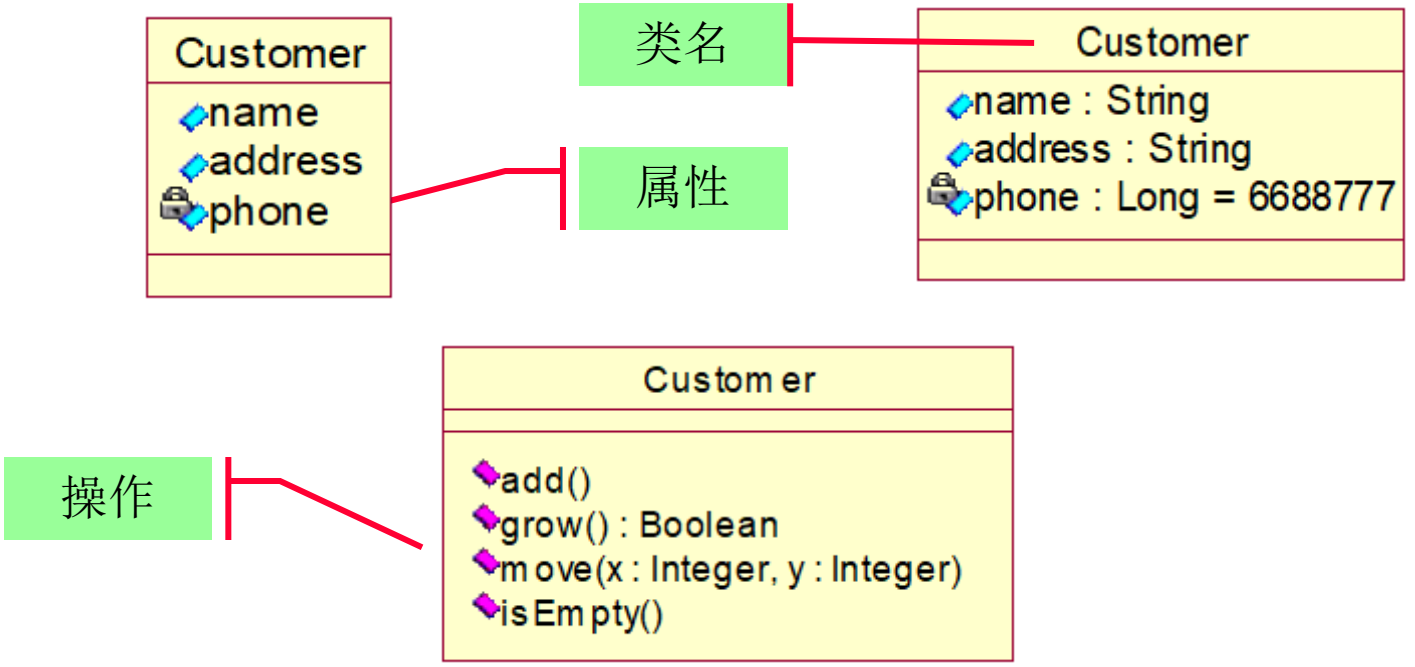
用例图示例

- 用例可以参与多种关系：关联、扩展、泛化和包含。



基本元素—类（class）

- 类是具有相同属性、操作和关系的对象集合的总称。
- 通常在UML中类被画成矩形，包括三个部分：
  - 名称：
  - 属性：
  - 操作：

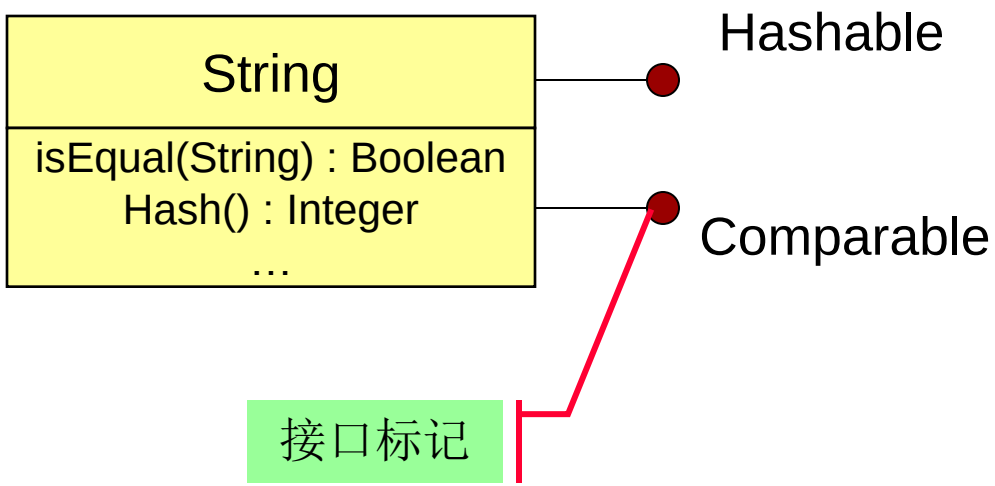


类图符的构成

- 名称：
  - 每个类都必须有一个名字，用于区别其它类。类名是一个字符串，称为简单名字。路径名字是在类名前加包含类的包名为前缀。例如Wall、java::awt::Wall都是合法的类名。
- 属性：
  - 类可以有任意多个属性，也可以没有属性。在类图中属性只要写上名字就可以了，也可以在属性名后跟上类型甚至缺省取值 。
- 操作：
  - 操作是类的任意一个实例对象都可以调用的，并可能影响该对象行为的实现。

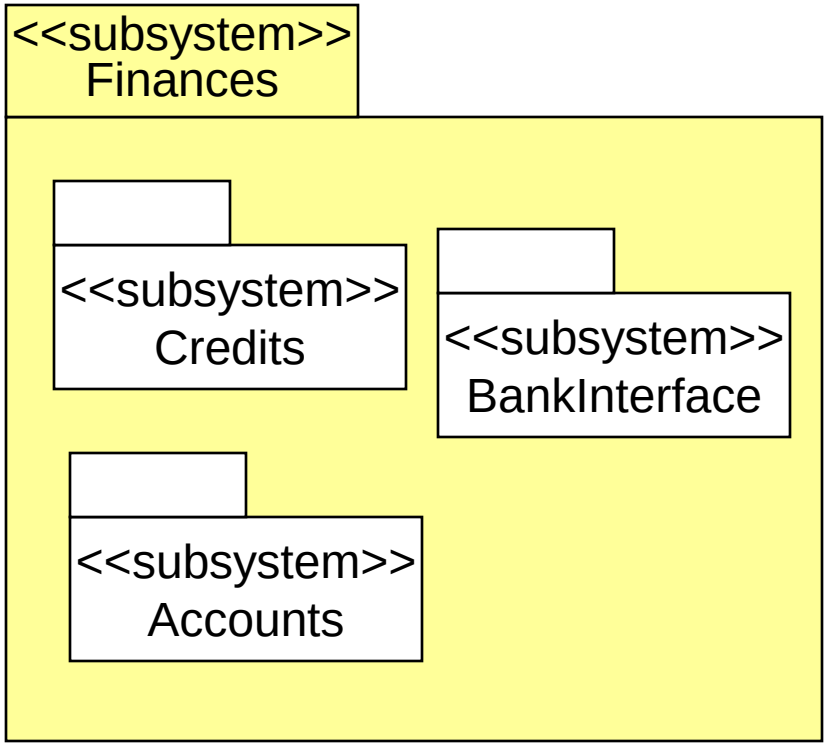
基本元素—接口（interface）

- 接口是未给出实现的对象行为的描述，接口包含操作，但没有属性，一个或多个类可以实现接口，每个类实现接口的操作。



基本元素—子系统（subsystem或package）

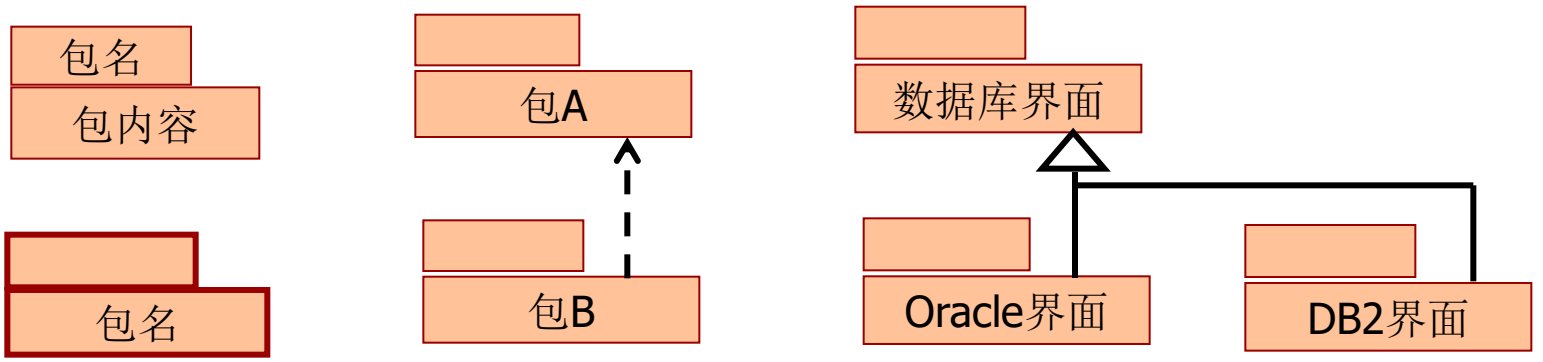
- OO解决问题的思路之一是将许多类集合成一个更高层次的单位，形成一个高内聚、低耦合的类的集合。UML中这种分组机制叫子系统（subsystem）或包(package)。
- 引入包是为了降低系统的复杂性，以便人们在某一时刻可以只处理有限的信息，使团队的工作不相互影响。



包的构成

- 包是一种组合机制，把各种各样的模型元素通过内在的语义连在一起成为一个整体就叫包，构成包的模型元素称为包的内容，包通常用于对模型的组织管理。
- 包可以包含各种模型元素和其它的包，如：可以是类的列表，也可以是另一个包图，还可以是一个类图。
- 包之间的关系有依赖和泛化（继承）。
- 包拥有自己的模型元素，包与包之间不能共用一个相同的模型元素，包的实例没有任何语义（含义）。仅在模型执行期间包才有意义。

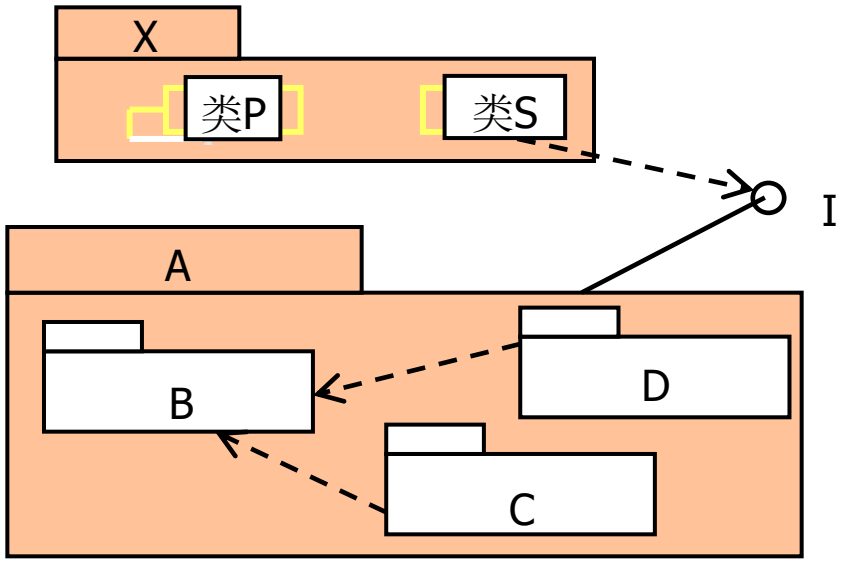
包之间的关系



（a）包的表示　(b)包的依赖关系　(c) 包的泛化关系

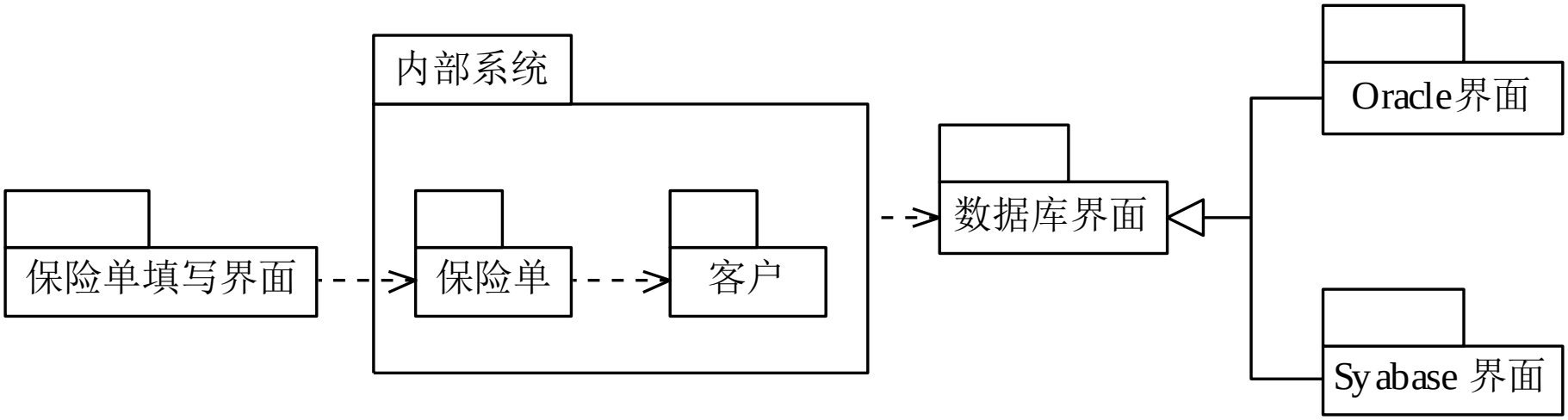
- 依赖关系：两个包中的任意两个类存在依赖关系，则包之间存在依赖关系。表示为：
- 泛化关系：使用继承中通用和特例的概念来说明通用包和专用包之间的关系。例如，专用包必须符合通用包的界面，与类继承关系类似。表示为：

包的可见性

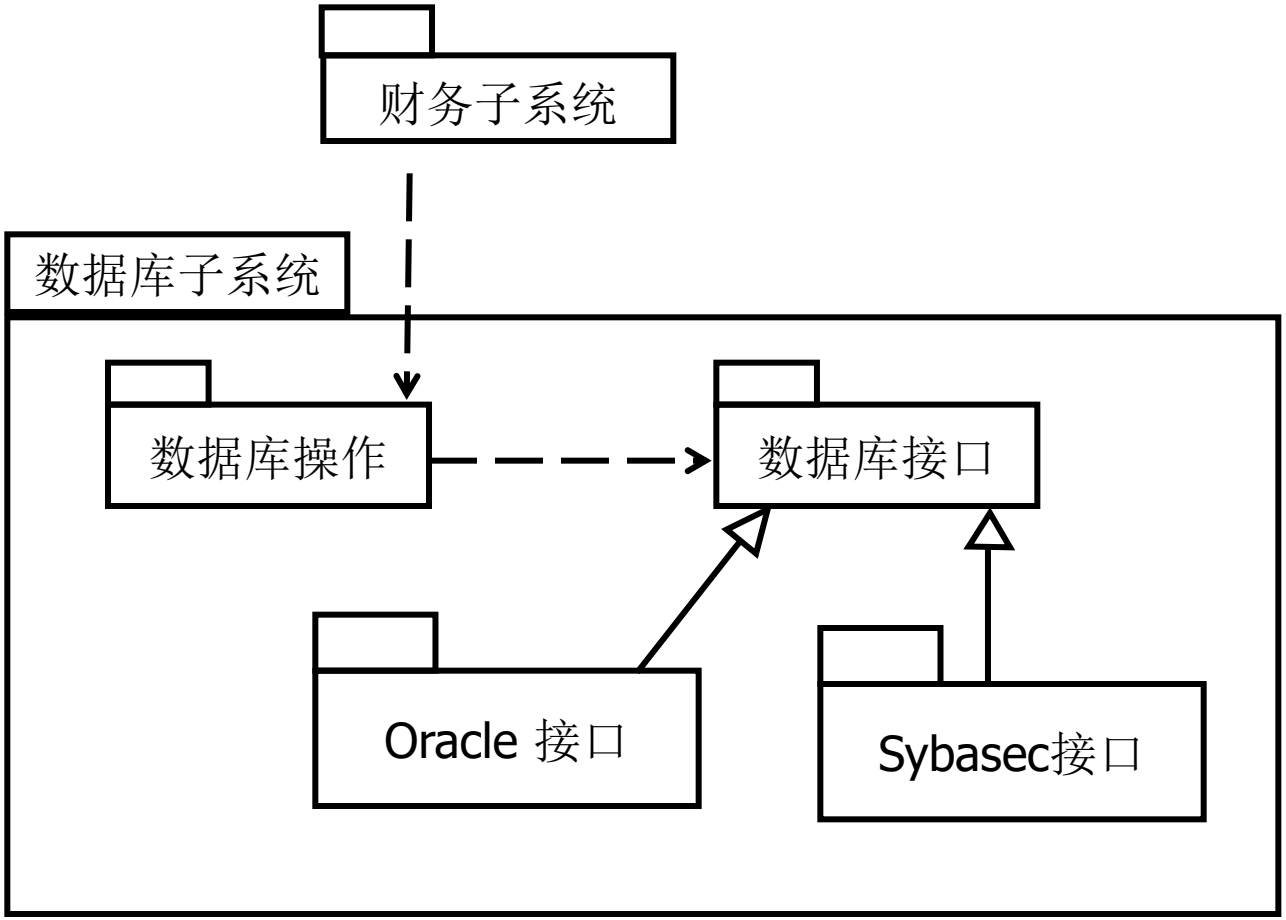


- 和类一样包也有可见性，利用可见性控制外部包对包的内容的存取方式，UML中定义了四种可见性：私有（private），公有（public），保护（protected）和实现（implementation）。缺省值为公有。
- 包也可以有接口，接口与包之间用实线相连，接口通常由包的一个或多个类实现。

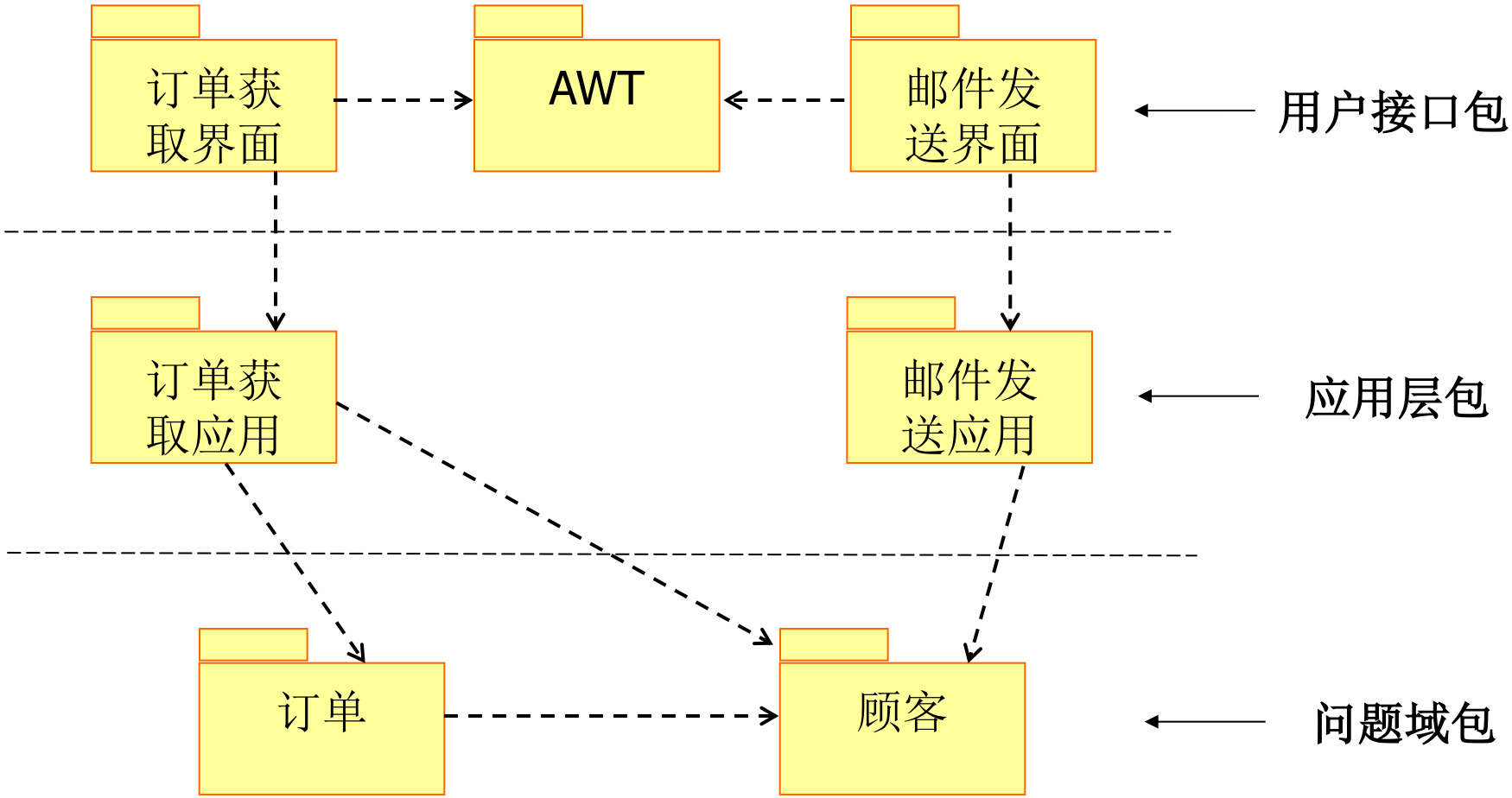
保险信息管理系统的包图



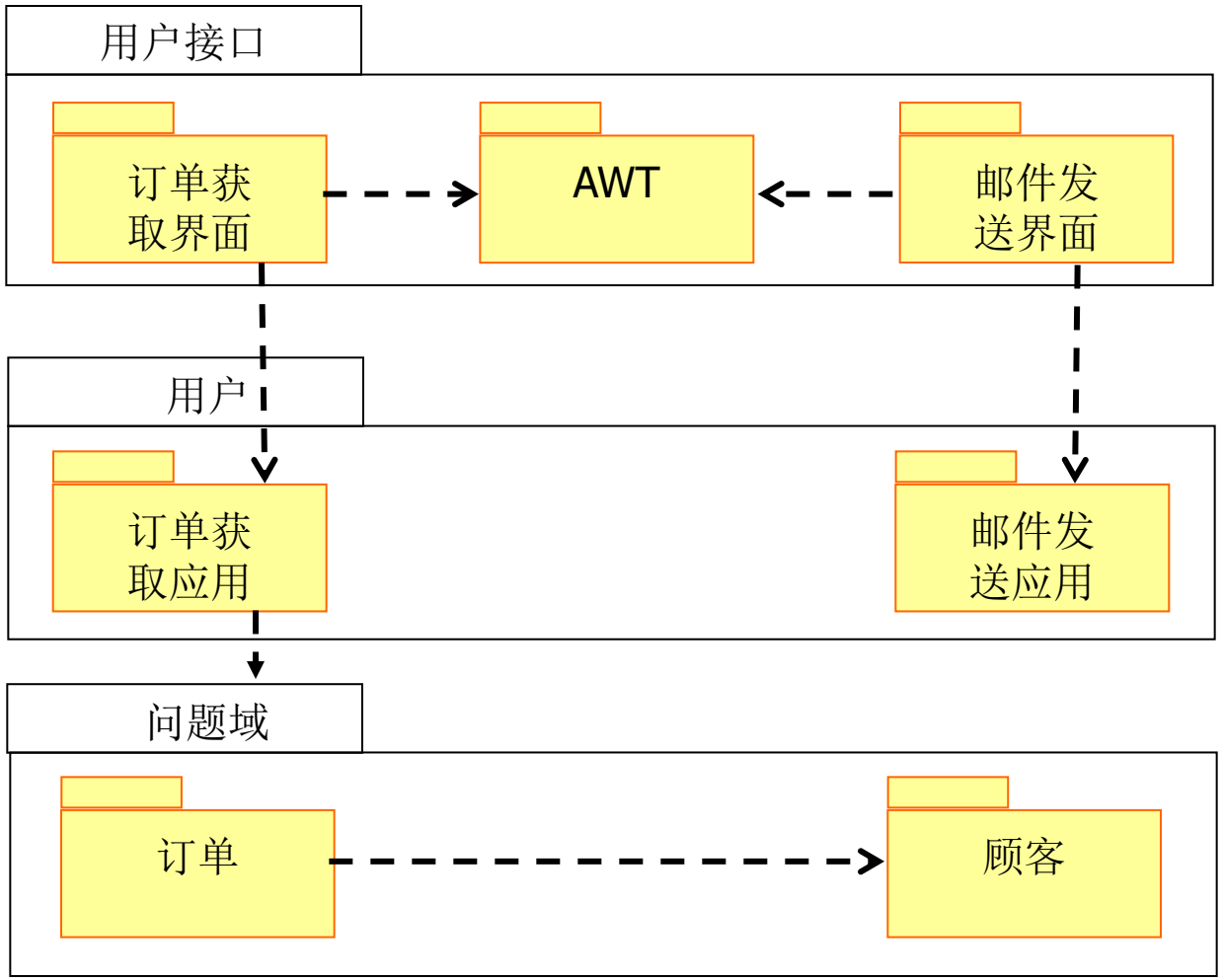
包图举例



（续）

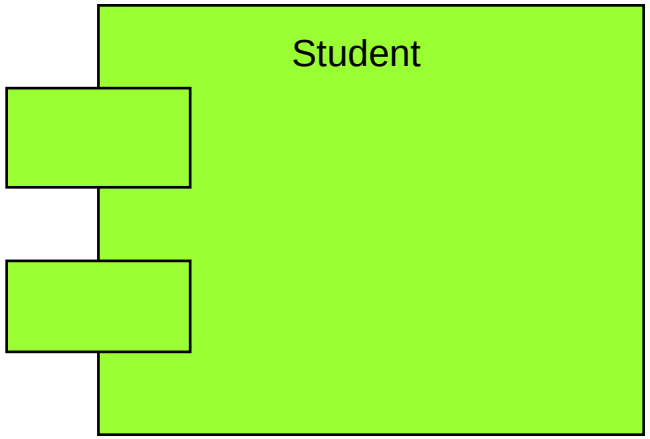


包图另一种划分方式



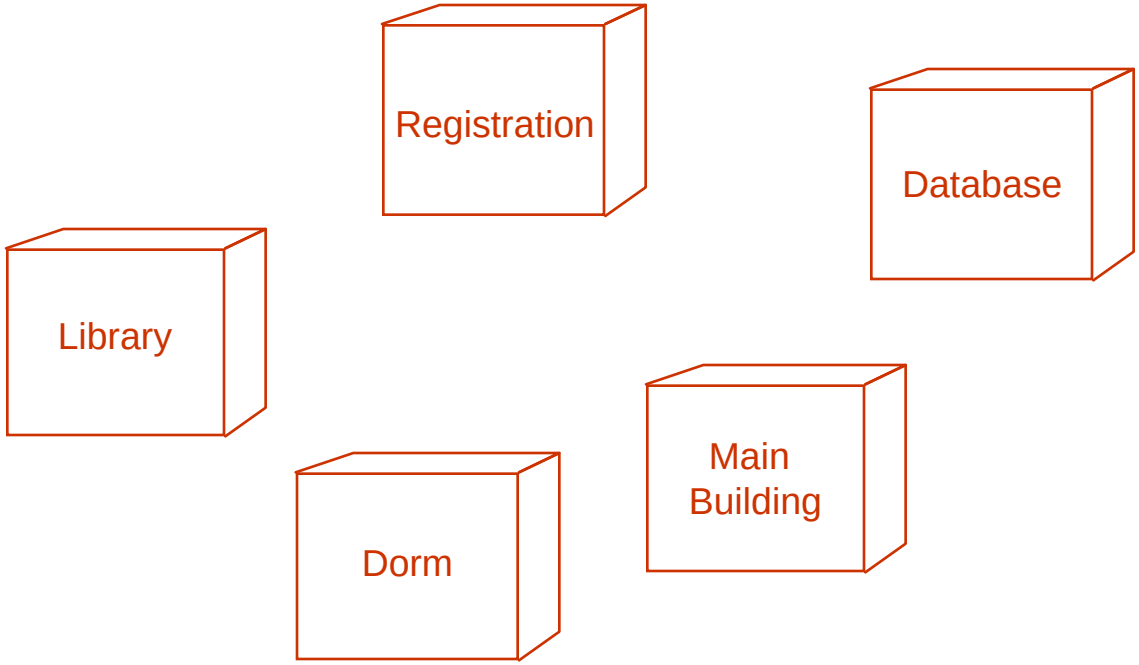
基本元素—组件（component）

- 组件是可重用的系统片段，具有良好接口定义的物理实现单元。每个组件包含了系统设计中某些类的实现。
- 组件设计的原则：良好的组件不直接依赖于其它组件的实现，只依赖于其它组件的接口。这样的好处是系统中的组件可以被支持相同接口的组件所取代。
- 组件可能是源代码、可执行程序或动态库。



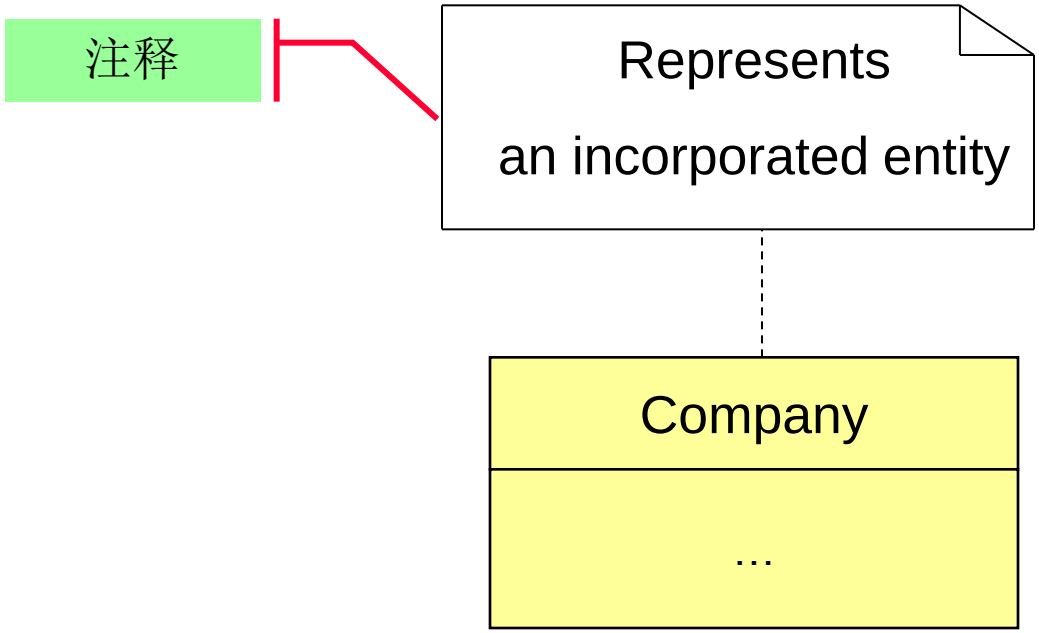
基本元素—结点（node）

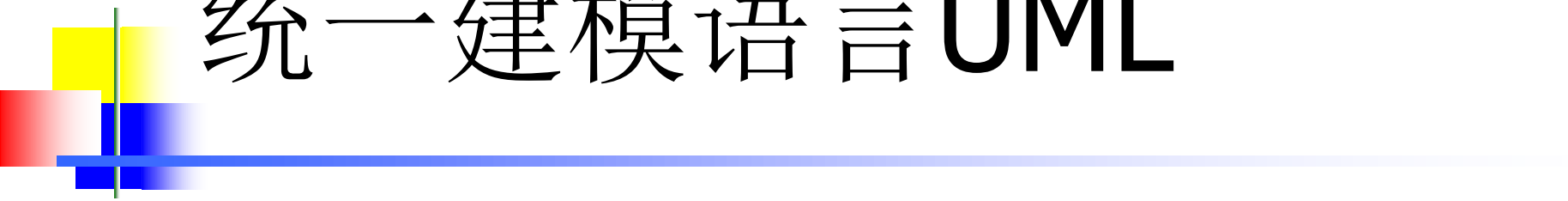
- 结点代表系统运行时的物理对象，结点通常拥有运算能力，它可以容纳对象和组件实例。



基本元素—注释（note）

- 注释用于解释设计的思路，便于理解。
- 一个好的模型应该有详尽的注释。





统一建模语言UML

4 静态建模

UML对静态建模的支持

- 所谓静态建模是指对象之间通过属性互相联系，而这些关系不随时间而转移。类和对象的建模，是UML建模的基础。
- 熟练掌握基本概念、区分不同抽象层次以及在实践中灵活运用，是三条最值得注意的建模基本原则。
- UML的静态建模机制包括：
  - 用例图(Use case diagram)
  - 类图(Class diagram)
  - 对象图(Object diagram)
  - 包图(Package diagram)
  - 构件图(Component diagram)
  - 配置图(Deployment diagram)

用例模型(use-case model)

- 用例模型描述了外部参与者（Actor）所理解的系统功能。
- 它描述了待开发系统的功能需求，它驱动了需求分析之后各阶段的开发工作 。
- 用例模型由若干个用例图构成，主要描述参与者和用例之间的关系。

创建用例模型的工作包括：

定义系统、确定执行者和用例、描述用例、定义用例间的关系、确认模型。

例：建立项目与资源管理系统的用例图

PRMS系统的主要功能是：

- 项目管理：包括项目的增加、删除、更新。
- 资源管理：包括对资源和技能的添加、删除和更新。
- 系统管理：包括系统启动和关闭，数据存储和备份等功能。

1. 分析确定系统的执行者（角色）

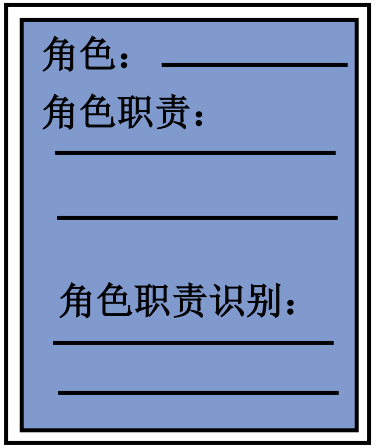
项目管理员、资源管理员、系统管理员、备份数据系统。

2. 确定用例

项目管理、资源管理和系统管理。

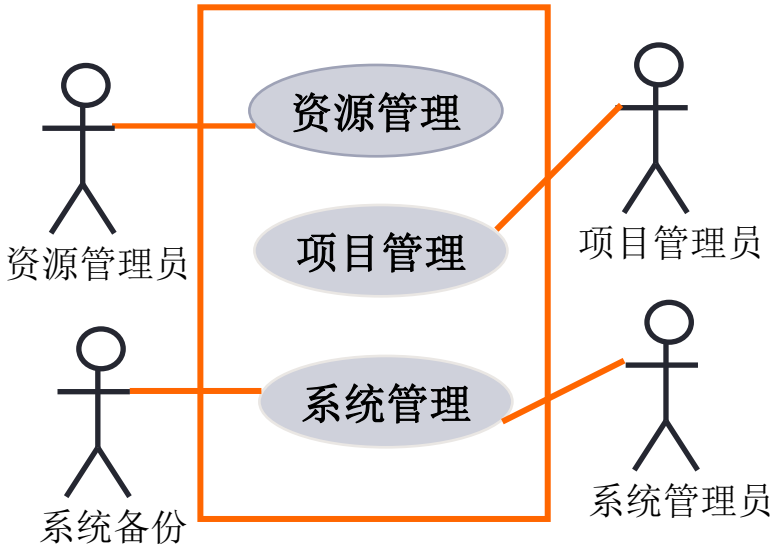
3. 对用例进行分解，画出下层的Use case图

对上层的用例进行分解，并将执行者分配到各层次的用例图中。还应画出相应的执行者描述模板及用例描述模板。

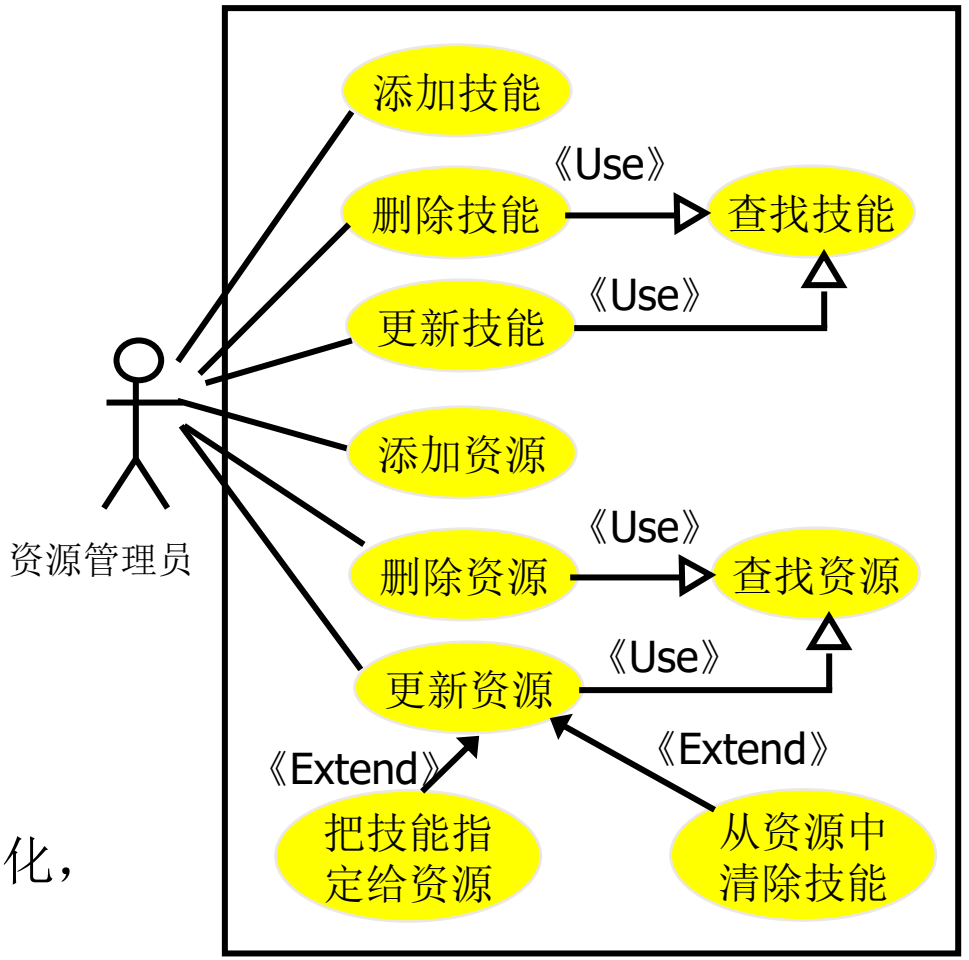


角色描述模板

例续：项目与资源管理系统（PRMS）



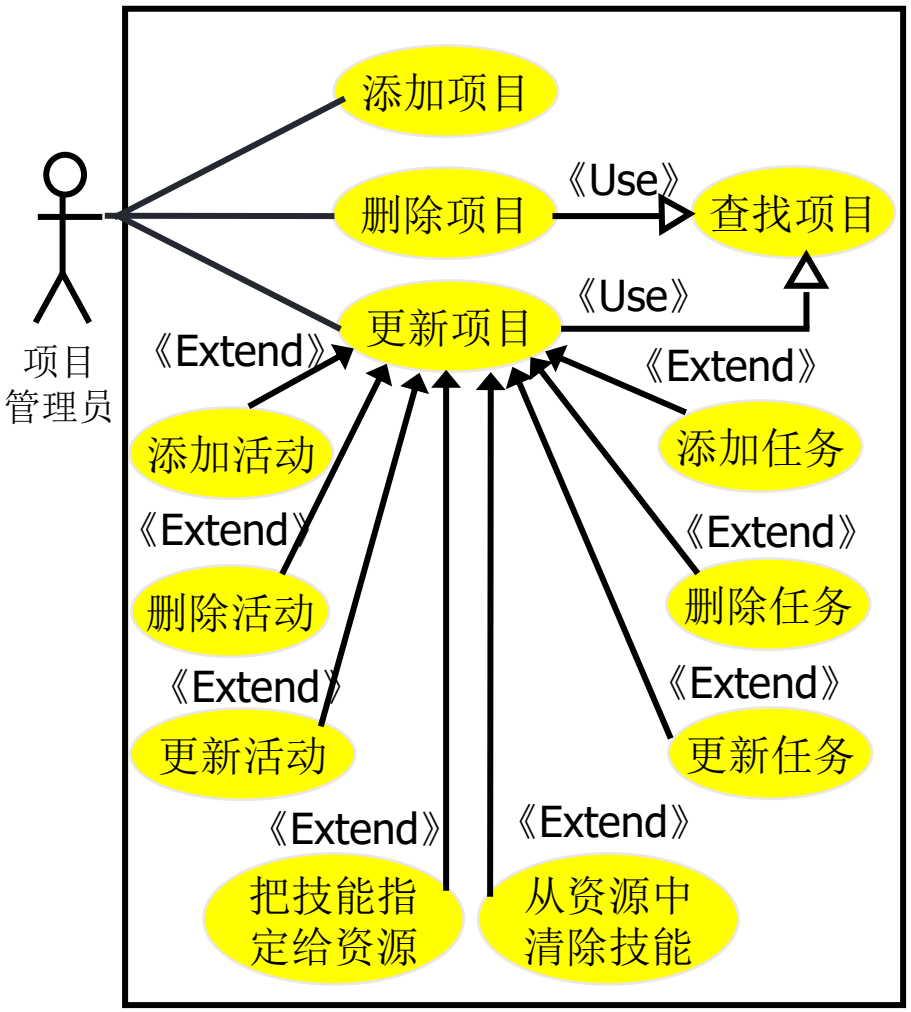
PRMS高层Use Case图



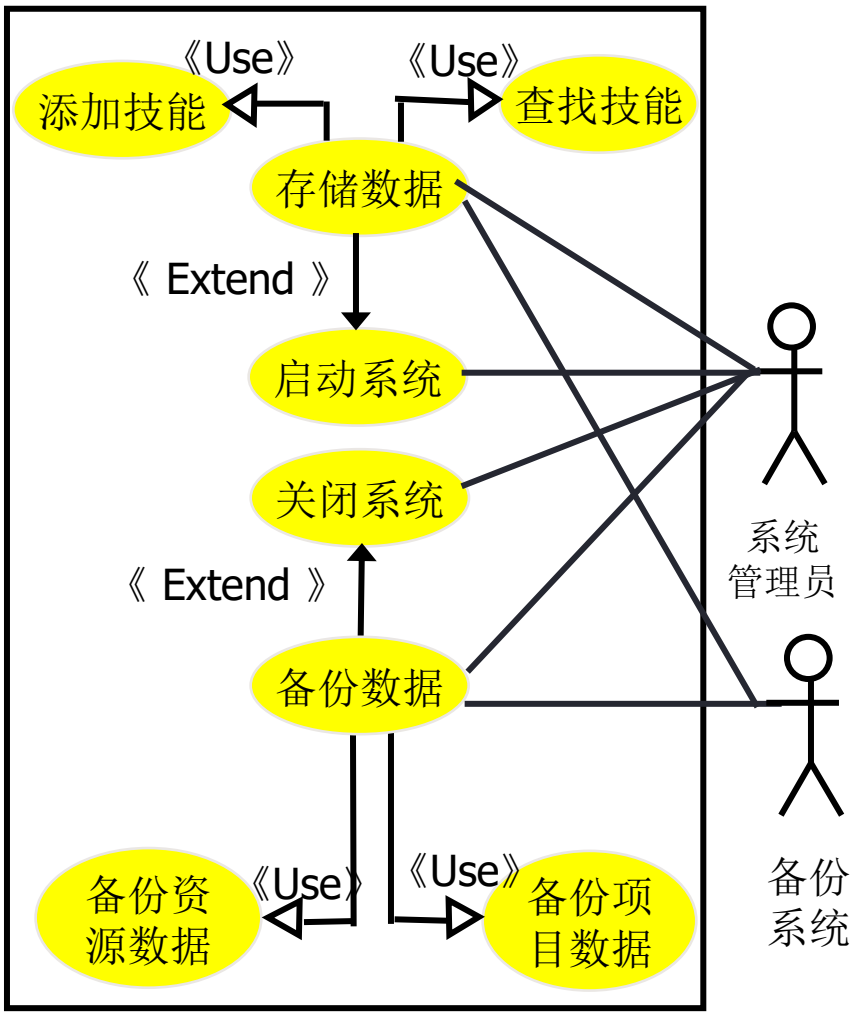
资源管理Use Case图

Use Case图可以自顶而下不断精化，抽象出不同层次的Use Case图。

例续：项目与资源管理系统（PRMS）



项目管理Use Case图



系统管理Use Case图

案例——医院病房监护系统

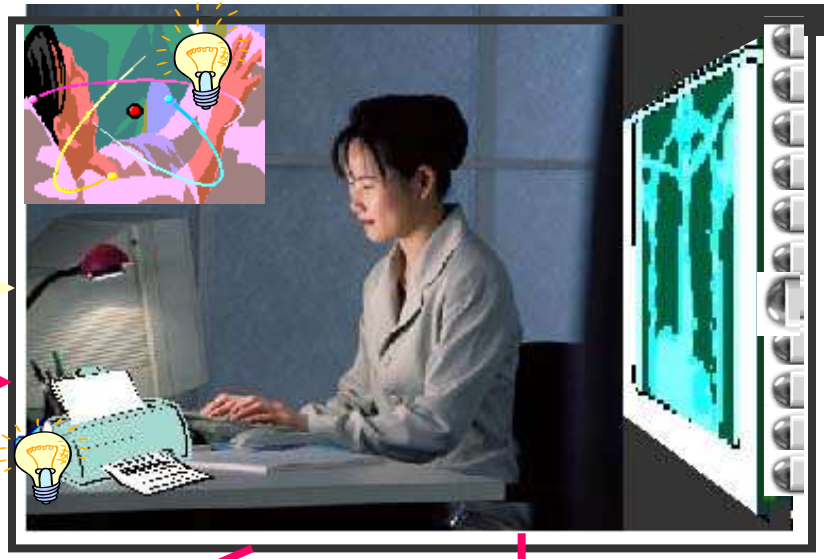
现有一医院病房监护系统，病症监视器安置在每个病房，将病人的病症信号实时传送到中央监视系统进行分析处理。在中心值班室里，值班护士使用中央监视系统对病员的情况进行监控，根据医生的要求随时打印病人的病情报告，定期更新病历，当病症出现异常时，系统会立即自动报警，并实时打印病人的病情报告，立及更新病历。

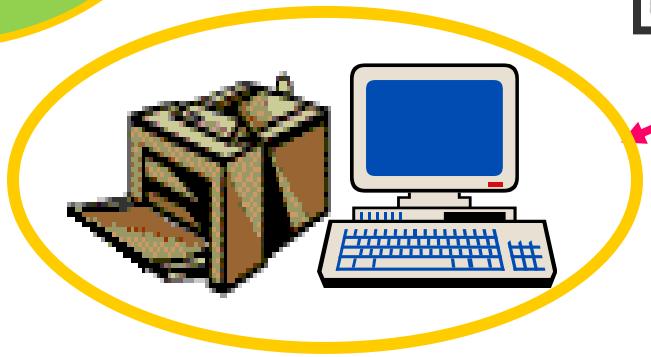
要求根据现场情景，对医院病房监护系统进行需求分析，建立系统的用例模型

医院病房监护系统



监视病情





产生
病情报告



更新病历

经过初步的需求分析，得到系统功能要求：

① 监视病员的病症（血压、体温、脉搏等）

② 定时更新病历

③ 病员出现异常情况时报警。

④ 随机地产生某一病员的病情报告。

案例——简单的需求分析说明

- 系统名称：**医院病房监护系统**
- 根据分析，系统主要实现以下功能：

①病症监视器可以将采集到的病症信号（组合），格式化后实时的传送到中央监护系统。

②中央监护系统将病人的病症信号开解后与标准的病症信号库里的病症信号的正常值进行比较，当病症出现异常时系统自动报警。

③当病症信号异常时，系统自动更新病历并打印病情报告。

④值班护士可以查看病情报告并进行打印。

⑤医生可以查看病情报告，要求打印病情报告，也可以查看或要求打印病历。

⑥系统定期自动更新病历。

案例——系统的静态建模

建立系统的用例图

- 通过以下六个问题识别角色

① 谁使用系统的主要功能？

② 谁需要系统的支持以完成日常工作任务？

③ 谁负责维护，管理并保持系统正常运行？

④ 系统需要应付（或处理）哪些硬设备？

⑤ 系统需要和哪些外部系统交互？

⑥ 谁（或什么）对系统运行产生的结果（值）感兴趣？

案例——系统的静态建模（续）

通过回答这六个问题以后，再进一步分析可以识别出本系统的四个角色：值班护士、医生、病人、标准病症信号库。

角色描述模板：

角色：病人
角色职责：
提供病症信号

角色职责识别：

负责生成、实时提供各种病症信号。

角色：医生
角色职责：
对病人负责，负责处理病情的变化
角色职责识别：
(1)需要系统支持以完成其日常工作
(2)对系统运行结果感兴趣

角色：值班护士
角色职责：
负责监视病人的病情变化
角色职责识别：
(1)使用系统主要功能
(2)对系统运行结果感兴趣

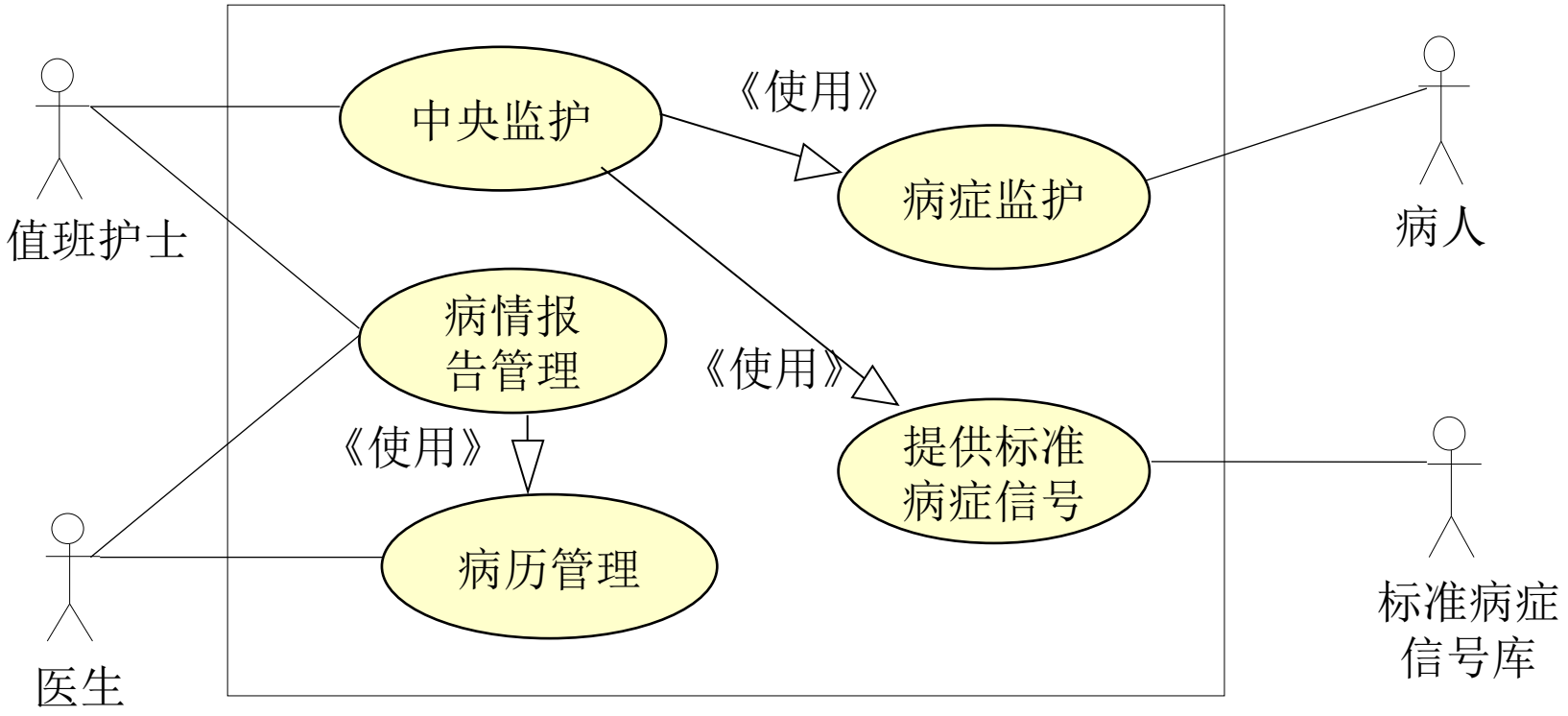
角色:标准病症信号库
角色职责：
负责向系统提供病症信号的正常值

角色职责识别：
(1)负责保持系统正常运行
(2)与系统交互

案例——系统的静态建模（续）

初步识别出系统的用例为：中央监护、病症监护、提供标准病症信号、病历管理、病情报告管理。

顶层用例图为：



案例——系统的静态建模（续）

将用例细化，可以得到分解的用例：

① 中央监护

a) 分解信号：将从病症监护器传送来的组合病症信号分解为系统可以处理的信号。

b) 比较信号：将病人的病症信号与标准信号比较。

c) 报警：如果病症信号发生异常（即高于峰值），发出报警信号。

d) 数据格式化：将处理后的数据格式化以便写入病历库。

② 病症监护

e) 信号采集：采集病人的病症信号。

f) 模数转化：将采集来的模拟信号转化为数字信号。

g) 信号数据组合：将采集到的脉搏，血压等信号数据组合为一组信号数据。

h) 采样频率改变：根据病人的情况改变监视器采样频率。

案例——系统的静态建模（续）

③ 提供标准病症信号

i) （此用例不分解）

④ 病历管理

j) 生成病历

k) 查看病历

l) 更新病历

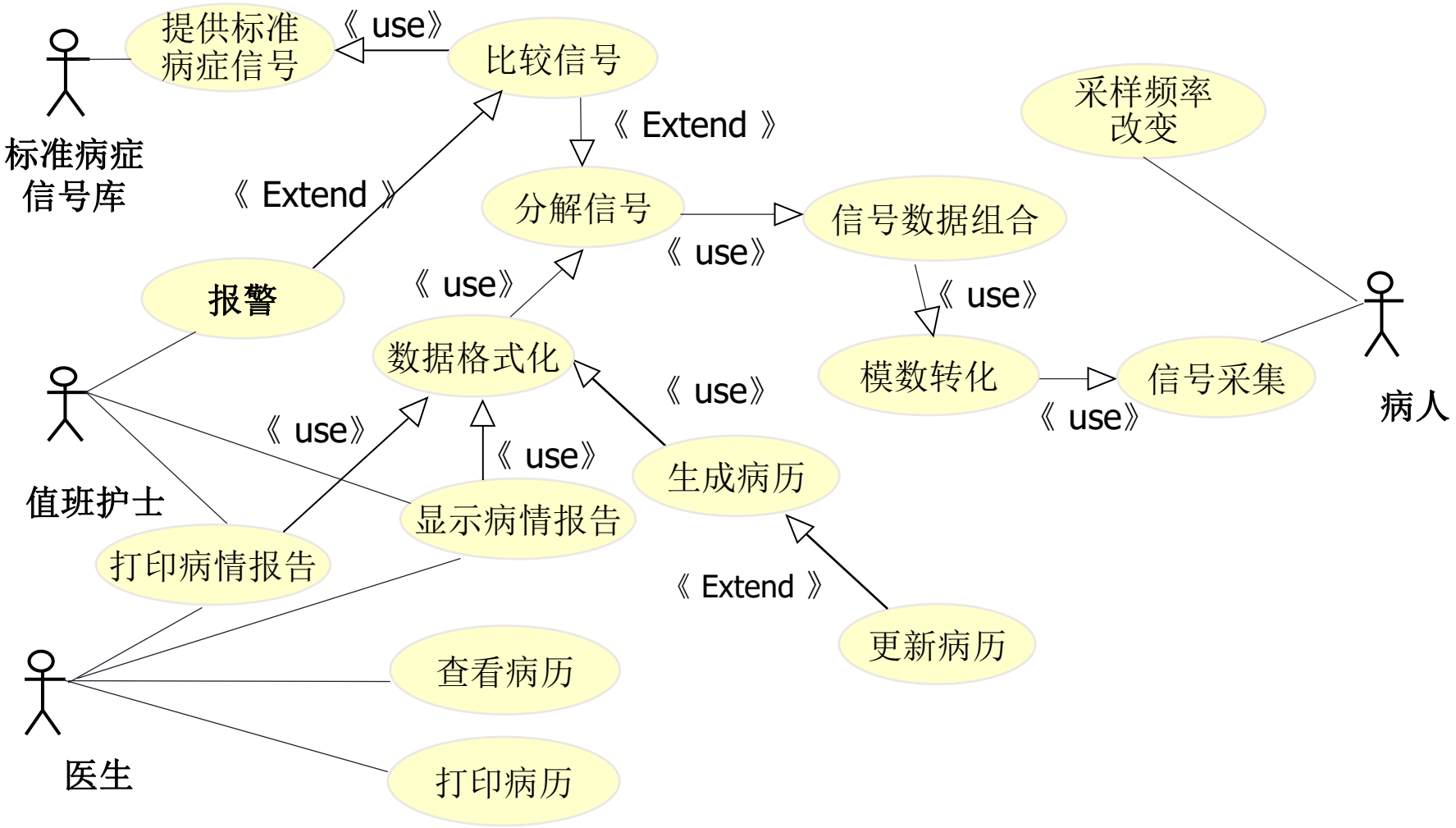
m) 打印病历

⑤ 病情报告

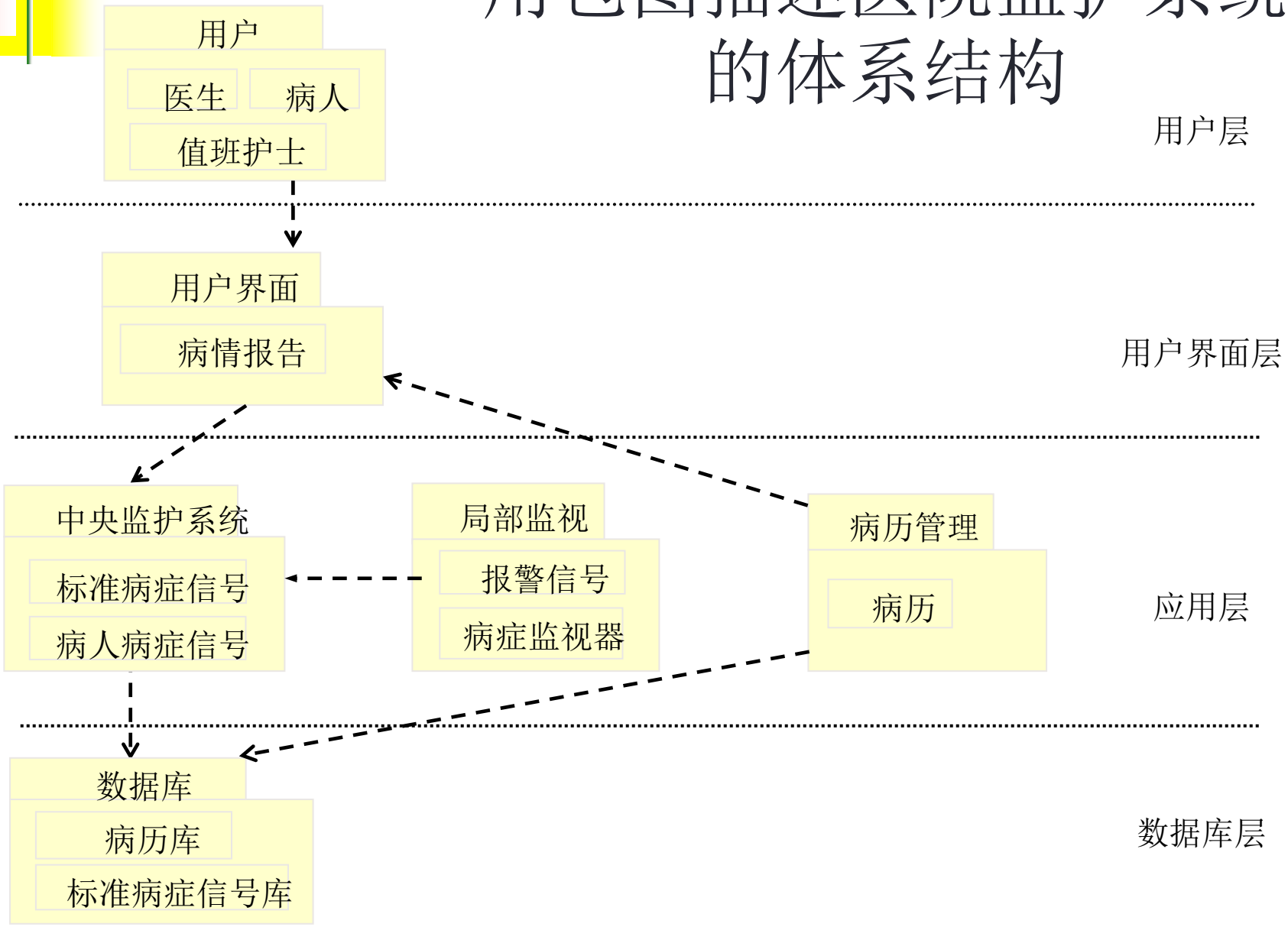
n) 显示病情报告：在显示器上显示病情

o) 打印病情报告：在打印机打印病情报告

案例——细化出的用例图

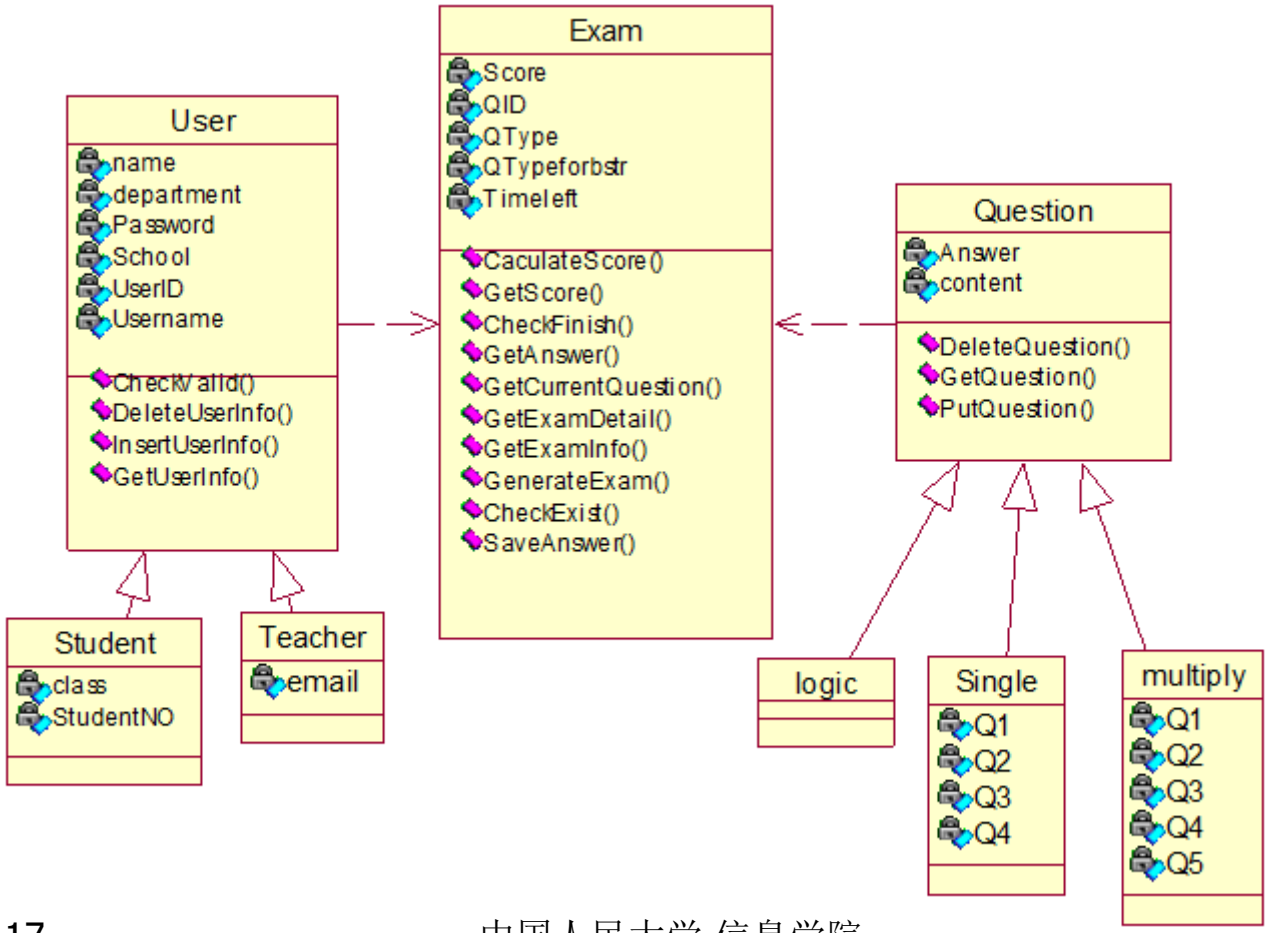

提供标准病症信号
《 use》
比较信号
标准病症信号库
《 Extend 》
《 Extend 》
分解信号
信号数据组合
采样频率改变
《 use》
《 use》
《 use》
报警
数据格式化
模数转化
信号采集
病人
《 use》
《 use》
《 use》
值班护士
生成病历
显示病情报告
打印病情报告
《 Extend 》
更新病历
查看病历
医生
打印病历

用包图描述医院监护系统的体系结构



类图（class diagram）

静态视图是UML的基础，静态视图表示为类图和对象图，主要是描述类和类之间的关系。



类图的描述

(1)属性（attribute）

- 属性用来描述类的特征，表示需要处理的数据。
- 属性定义：

  visibility attribute-name : type = initial-value {property-string}

  可见性　　属性名：类型 = 缺省值 { 约束特性 }

- 其中：可见性(visibility)表示该属性对类外的元素是否可见。如果可见性未申明，表示其可见性不确定。分为：
  - public（+） 公有的，即模型中的任何类都可以访问该属性。
  - private（-） 私有的，表示不能被别的类访问。
  - protected（#） 受保护的，表示该属性只能被该类及其子类访问。

类图的描述（续）

(2) 操作

- 对数据的具体处理方法的描述则放在操作部分，操作说明了该类能做些什么工作。操作通常称为函数，它是类的一个组成部分，只能作用于该类的对象上。
- 操作定义：

visibility operating-name(parameter-list): return-type {property - string}

可见性 操作名（参数表）：返回类型{约束特性}

- 其中：可见性同上。
- 参数表：参数名：类型，…

Parameter-name : type = default - value

返回类型：操作返回的结果类型。

对象图（object diagram）

对象图是系统在某一时刻的快照（snapshot）。

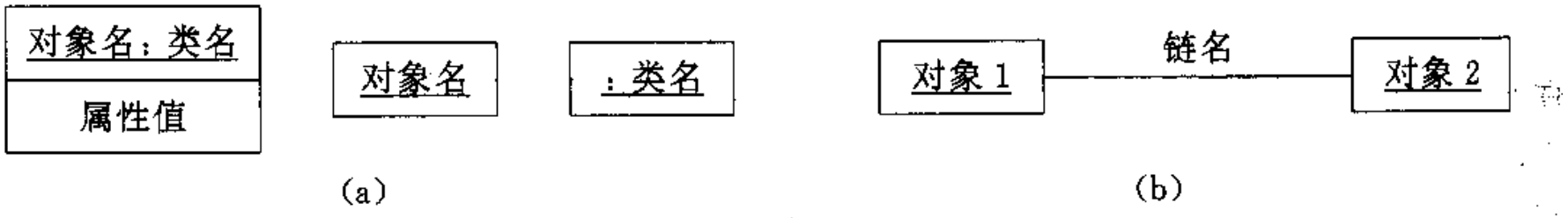
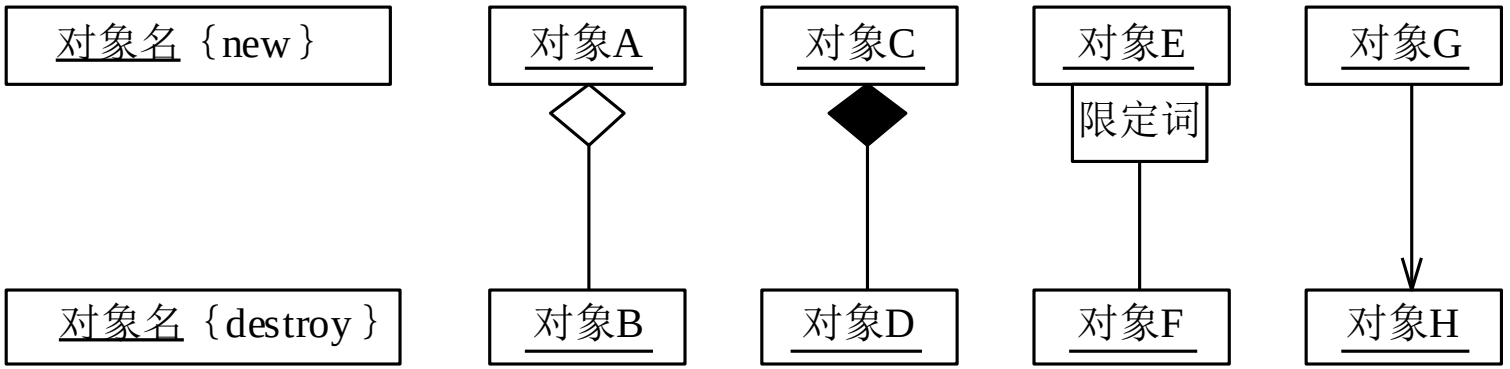


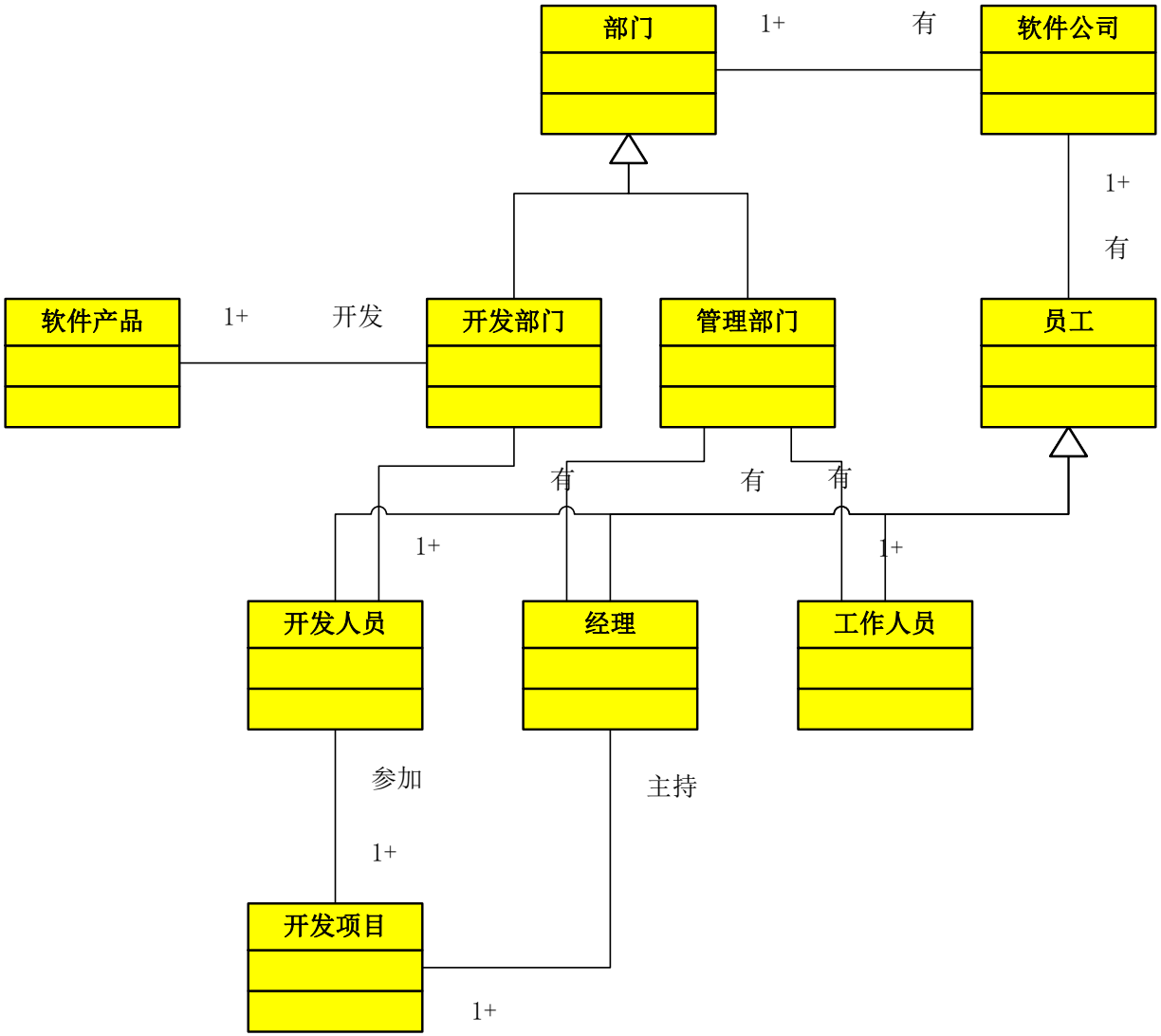
图 13.7　对象图的元素

(a) 对象的表示；(b) 链的表示



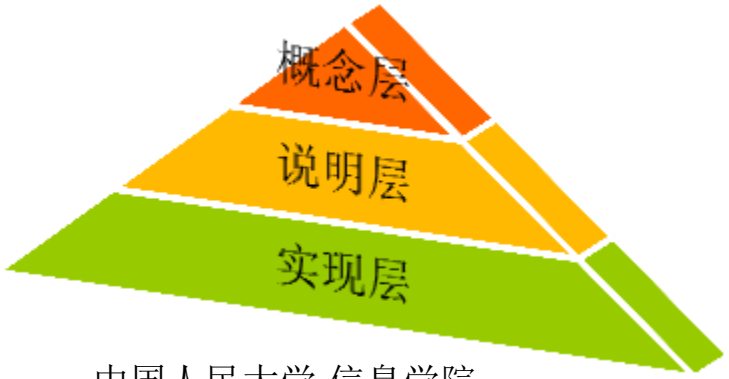
(a) 对象的创建和删除；　　(b) 各种关系的连接

例：软件公司的类图


部门
1+
有
软件公司
1+
有
软件产品
1+
开发
开发部门
管理部门
员工
有
有
有
1+
1+
开发人员
经理
工作人员
参加
1+
主持
开发项目
1+

类图的抽象层次和细化(Refinement)关系

- 在软件开发的不同阶段都使用类图，但这些类图表示了不同层次的抽象：
  - 在需求分析阶段，类图是研究领域的概念；
  - 在设计阶段，类图描述类与类之间的接口；
  - 在实现阶段，类图描述软件系统中类的实现。
- 按照Steve Cook和John Daniels的观点，类图分为三个层次：
  - 概念层
  - 说明层
  - 实现层



实例：识别系统的类

通过名词识别法和系统实体识别法等方法可以识别出系统的12个类，以下用类图的方法分别表示出类的名称，属性/操作。

值班护士	医生	病人	病症监视	中央监护系统	报警信号
标准病症信号库	病历	病历库	病人病症信号	病情报告	标准病症信号

值班护士
用户名 密码
查看病情报告（） 打印病情报告（）

医生
用户名 密码
查看病情报告（） 要求打印病情报告（） 查看病历（） 要求打印病历（）

病人
姓名 性别 年龄 病症
提供病症信号（）

病症监视
采集频率 病症信号
格式化信号数据（） 采集信号（） 信号组合（）

（续）

中央监护系统
输入 输出
分解信号（） 比较信号（） 报警（） 数据格式化（）

报警信号
声音 灯光 文字
报警（） 数模转化（）

标准病症信号库
类型 大小 容量
提供标准信号（）

病历库
类型 大小 容量
生成病历（） 更新病历（） 查看病历（） 打印病历（）

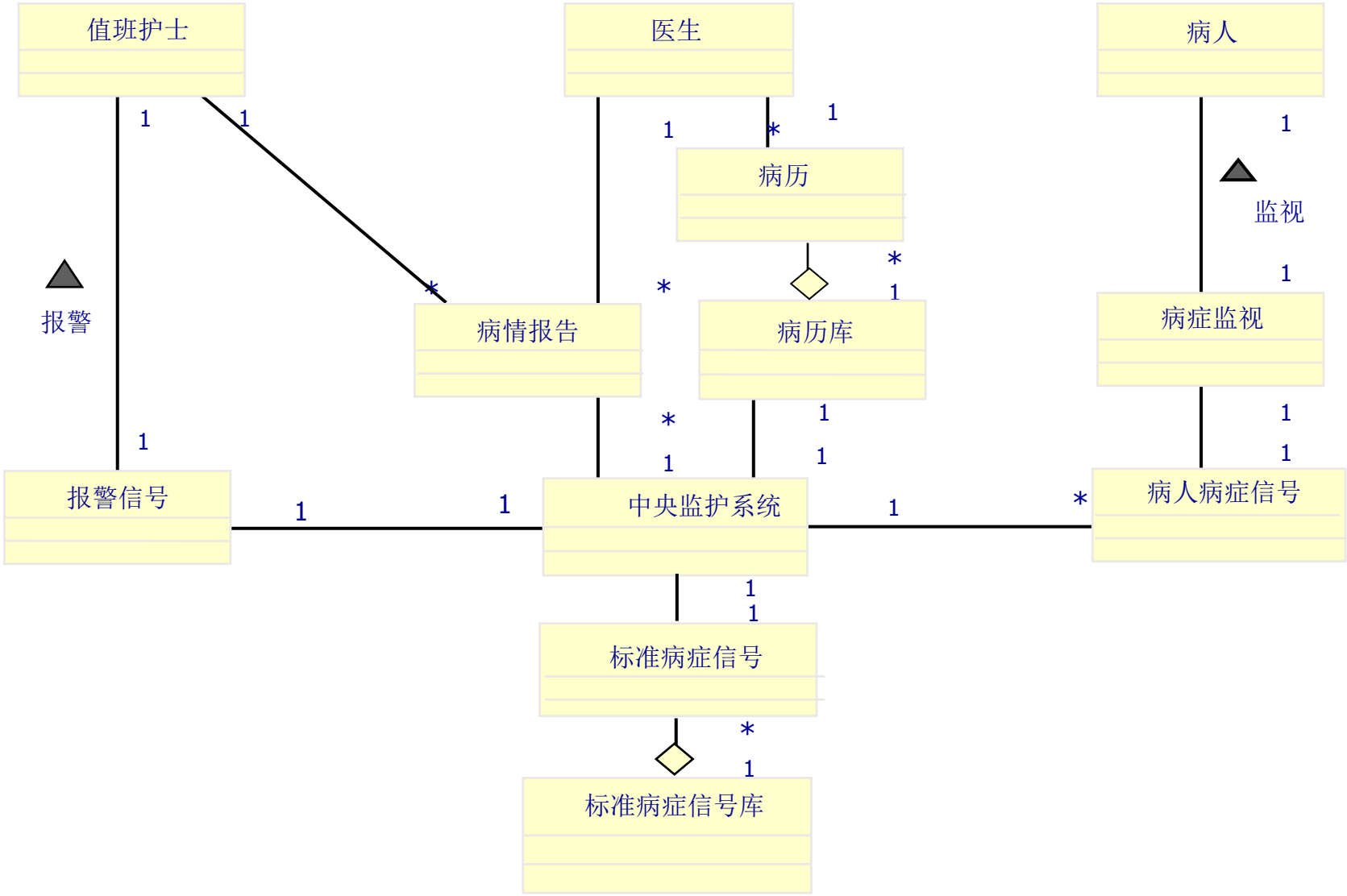
病人病症信号
脉搏 血压 体温
生成病症信号（）

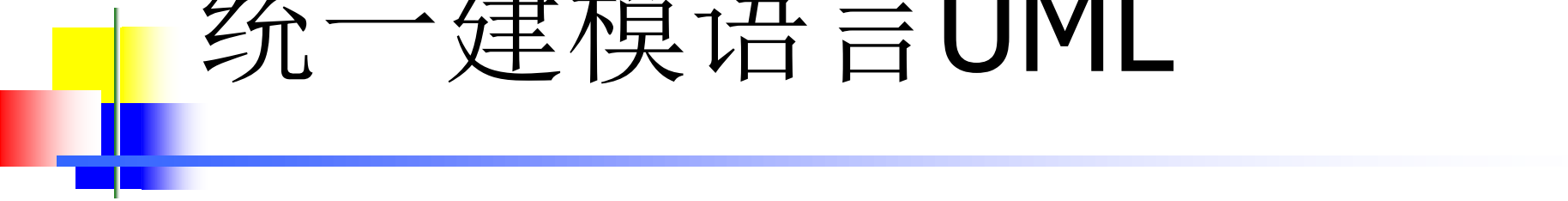
病情报告
标题 格式
生成病情报告（） 查看病情报告（） 打印病情报告（）

病历
格式 病人基本情况 打印时间
生成病历（） 查看病历（） 打印病历（）

标准病症信号
脉搏 血压 体温
生成标准信号（）

在类图中标明类之间的关系：





统一建模语言UML

5 动态建模

动态建模

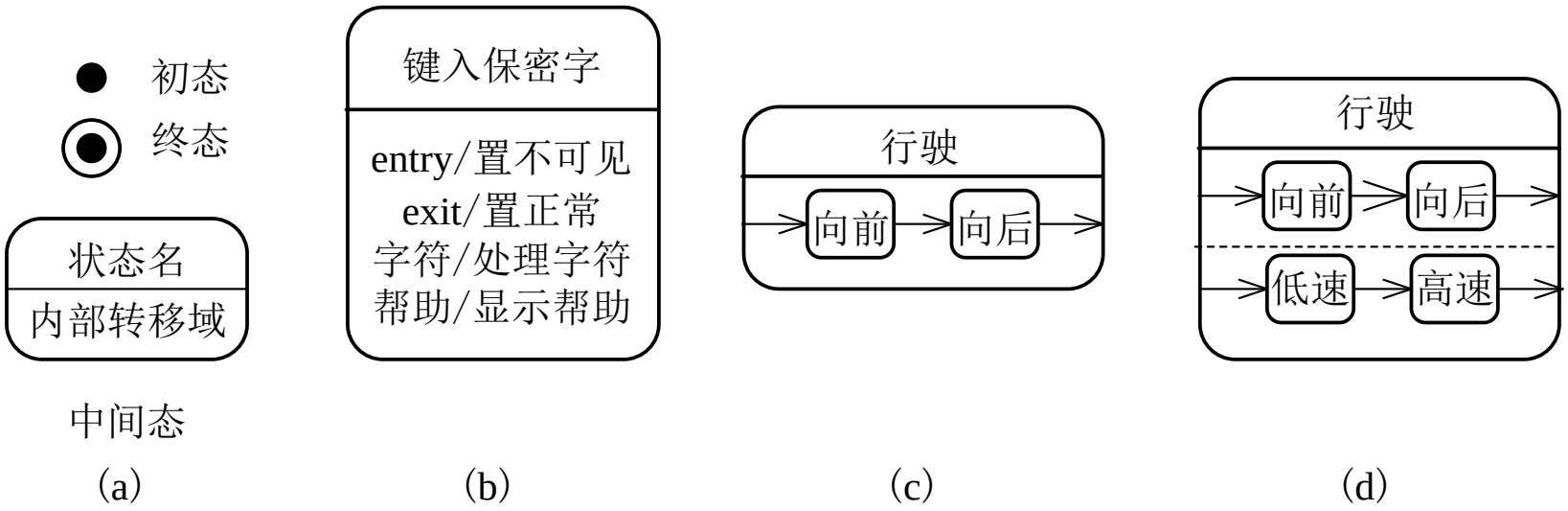
- 系统中的对象在执行期间的不同时间点如何通信以及通信的结果如何，就是系统的动态行为，也就是说，对象通过通信相互协作的方式以及系统中的对象在系统生命期中改变状态的方式，是系统的动态行为。
  - 状态图
  - 活动图
  - 顺序图
  - 协作图



状态图（state diagram）

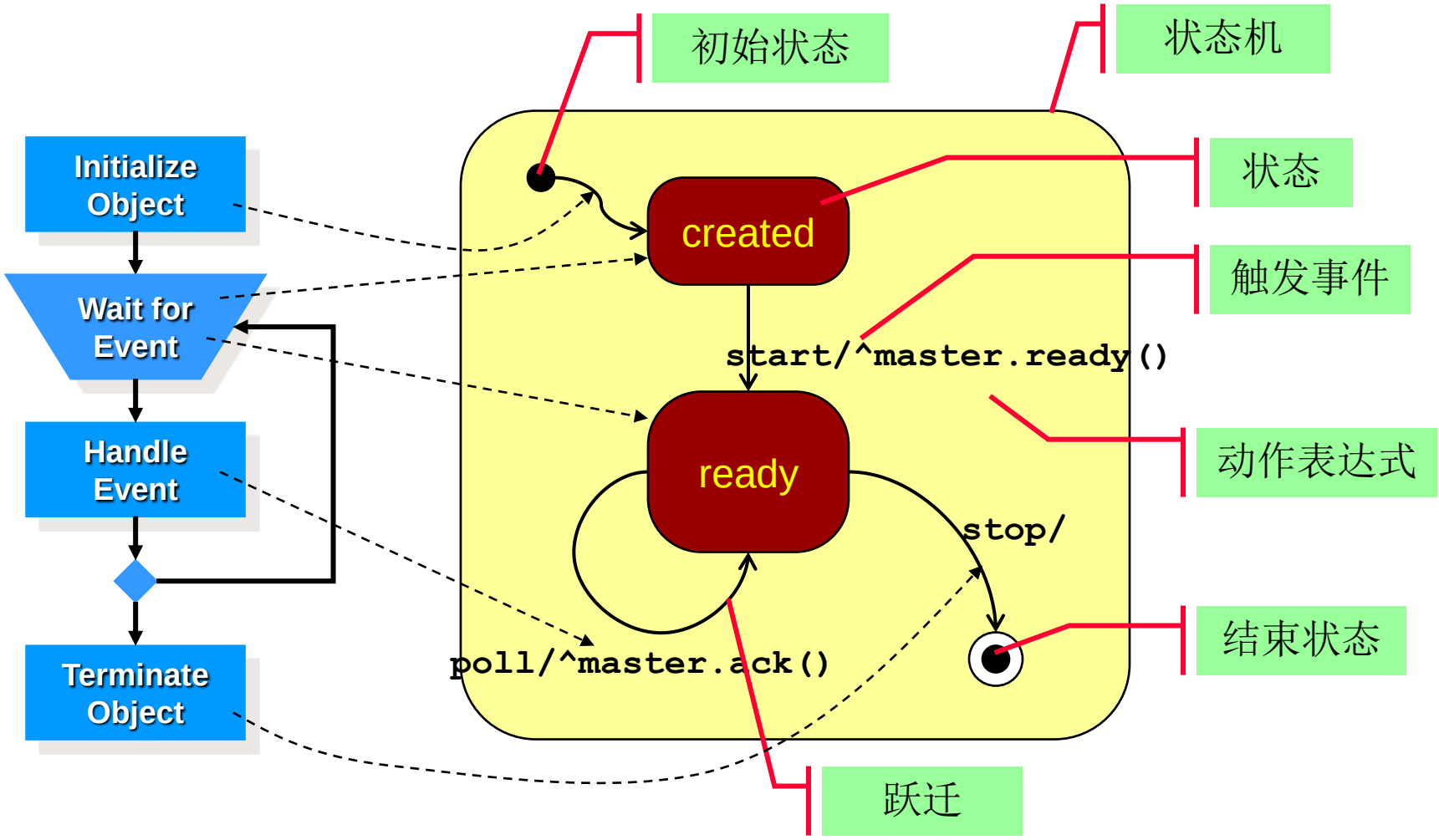
- 状态图是对单个类的对象的生命周期进行建模，描述了对象时间上的动态行为，每个对象被认为是事件驱动的孤立实体。
- 状态图是由状态（state）和跃迁（transition）组成的图，通常状态机附属于类，描述类实例对接受事件的响应。
- 事件（event）表达对象间的调用、显式信号、值的改变或时间的推移。
  - 调用事件、变更事件、信号事件、时间事件
- 状态（state）描述对象生命周期的一段时间，可以是等待其它事件时所处的时间，或是执行某一活动时所处的时间，状态分为简单状态和复合状态。
- 跃迁定义对象对某一事件的反应，通常，跃迁具有触发事件、跃迁条件、动作和目标状态。
- 跃迁的种类有外部跃迁和内部跃迁。外部跃迁是最普通的跃迁，会导致状态改变；内部跃迁不发生状态改变。
- 跃迁有两个隐式动作：进入动作（entry）和退出动作（exit）。无论何时进入和退出时都要执行，这方便进入时进行初始化工作，退出时进行资源的释放工作。

状态的表示

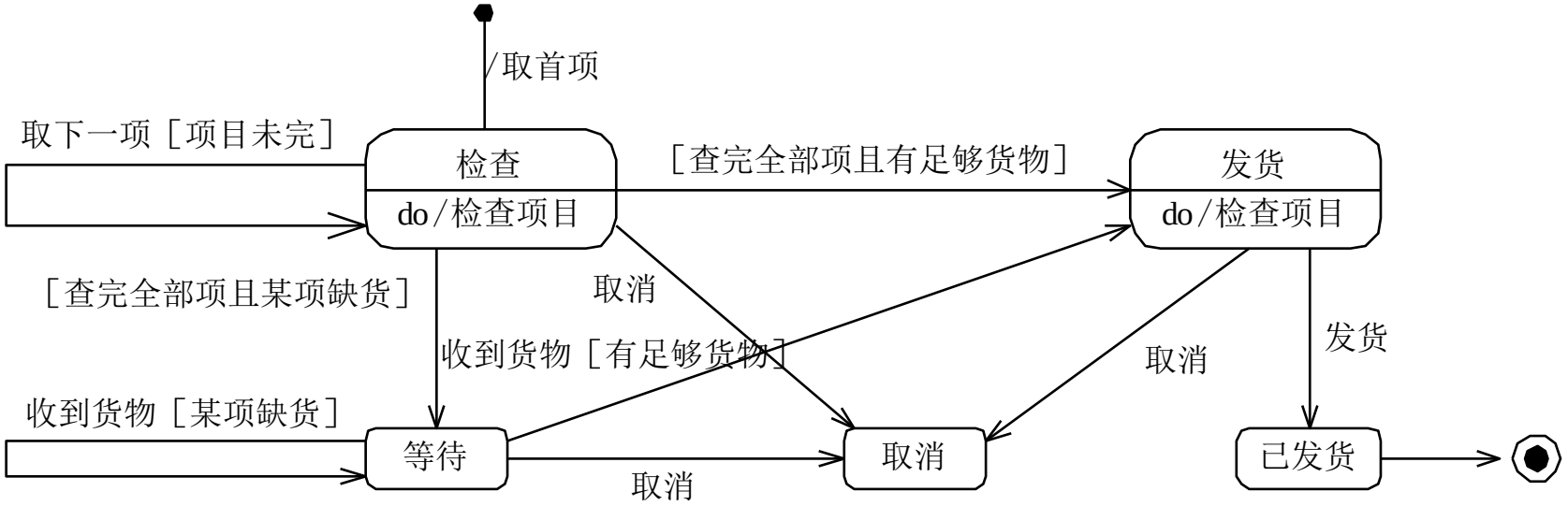


(a) 状态类型； (b) 内部转移域； (c) 或关系子状态； (d) 与关系子状态

状态图示例


初始状态
状态机
状态
触发事件
动作表达式
结束状态
跃迁
Initialize Object
Wait for Event
Handle Event
Terminate Object
created
ready
start/^master.ready()
stop/
poll/^master.ack()

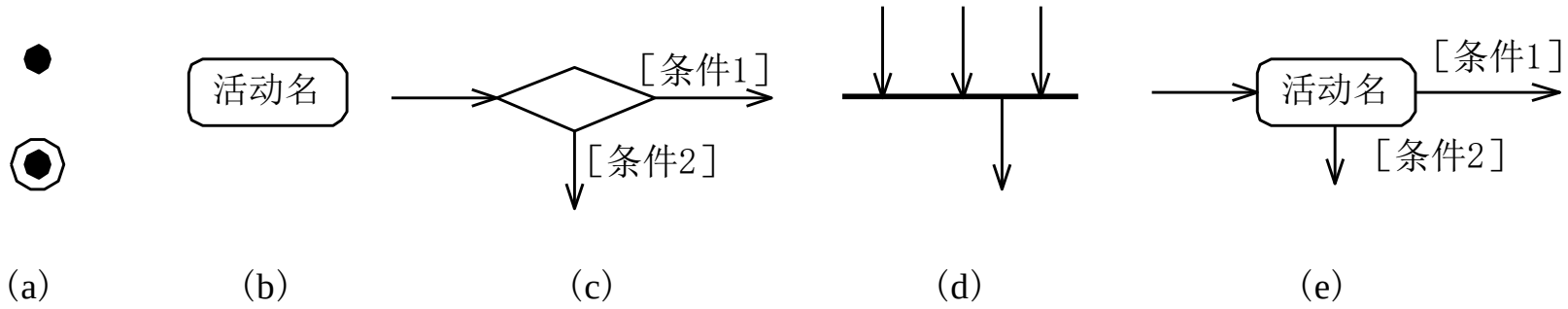
例：订单类的状态图


/取首项
取下一项［项目未完］
检查
do/检查项目
［查完全部项目且有足够货物］
发货
do/检查项目
［查完全部项目且某项缺货］
取消
收到货物［有足够货物］
取消
发货
收到货物［某项缺货］
等待
取消
取消
已发货

活动图（activity diagram）

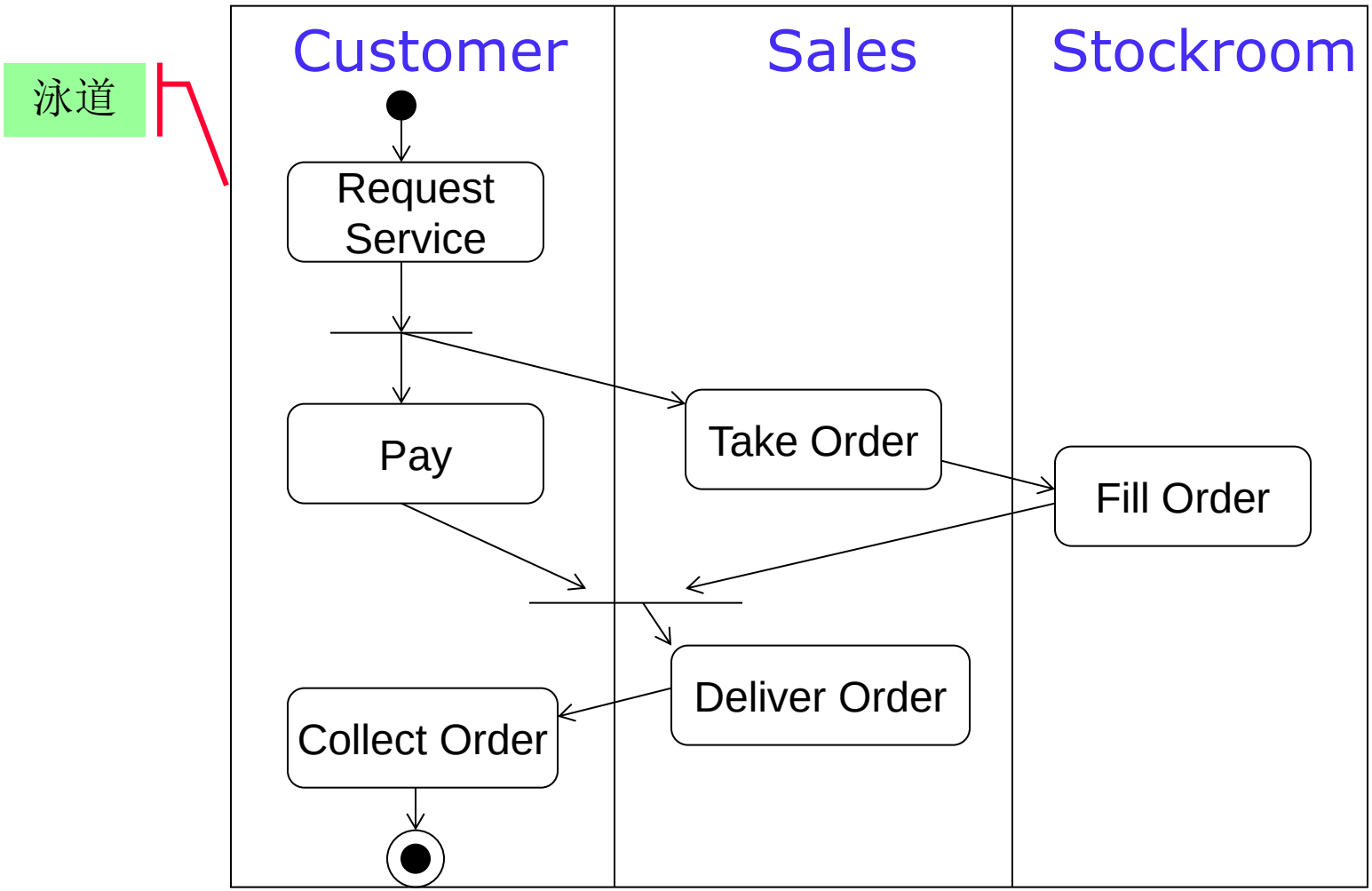
- 活动图是用状态机对工作流进行建模的特殊形式，它和流程图很类似，不过它支持并发控制。
- 活动图一般不描述所有的运算细节，它显示活动的流，但不显示执行活动的对象。
- 活动图处于系统的外部和内部视图之间，所以它可以作为设计的起点，为了完成设计，每个活动必须扩展成一个和多个操作，每个操作被指派给特定的对象来实现。
- 将商业组织控制的活动划分在一起，这类划分可以通过分隔的区域来表达，由于它们的外观，每个区域称为**泳道**（swim lane）。

活动图的元素



(a) 起点、终点； (b) 活动； (c) 判断； (d) 同步线； (e) 转移

活动图示例


泳道
Customer
Sales
Stockroom
Request Service
Pay
Take Order
Fill Order
Deliver Order
Collect Order

例：用活动图描述“找饮料”

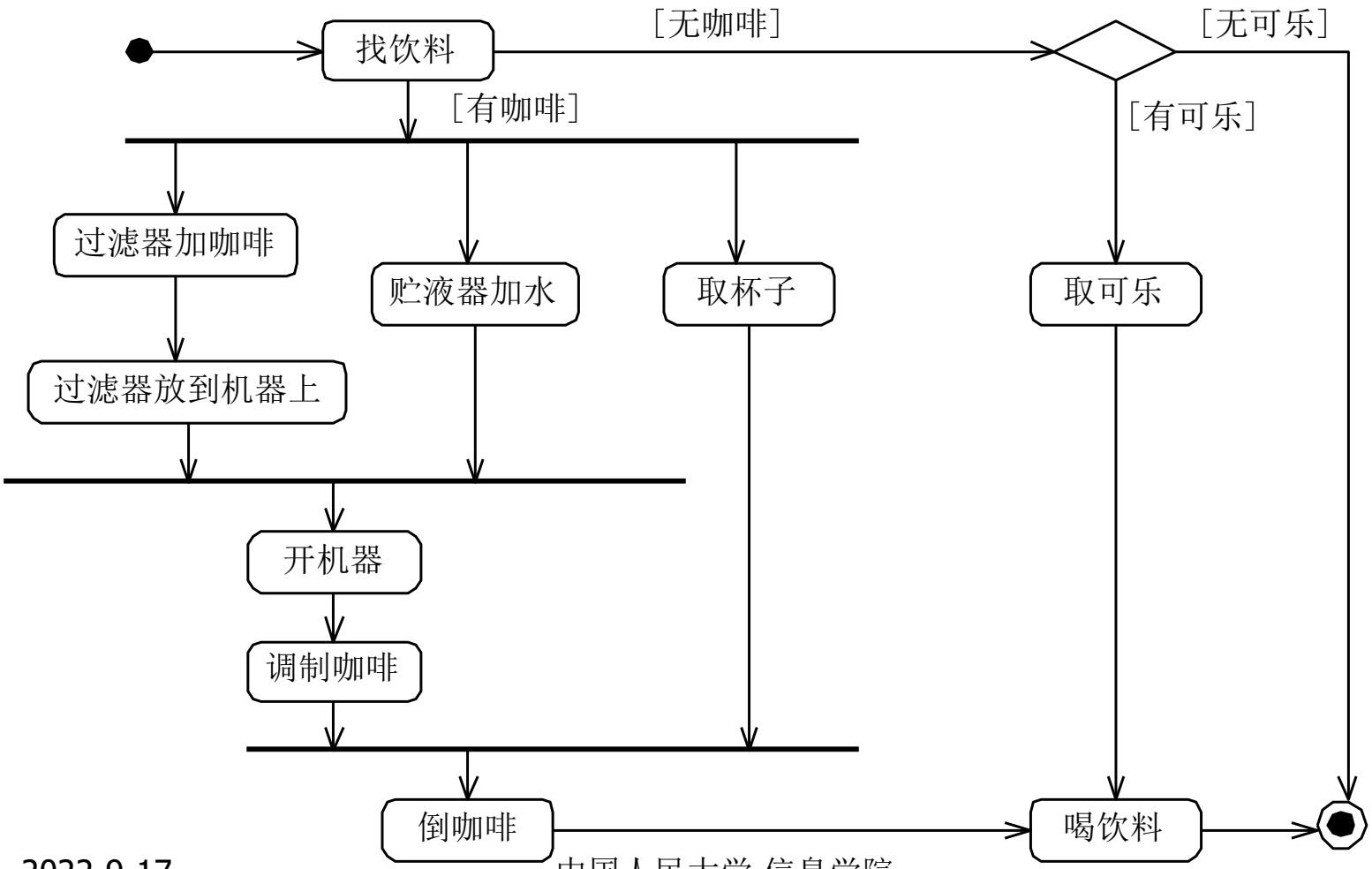
- 在“人”的类中关于“找饮料喝”方法的活动图。
- 从开始顺序转移到“找饮料”活动，该活动有两个转移。
  - ［有咖啡］时， 转移到一个同步线，同时开始 3 个活动：
    - 第一个是“过滤器加咖啡”活动，然后顺序转移到“过滤器放到机器上”活动;
    - 第二个是“贮液器加水”活动;
    - 第三个是“取杯子”活动。
  - ［无咖啡］时，则转移到有无可乐的判断。
  - ［有可乐］条件时，则执行：
    - “取可乐”活动
    - “喝饮料”活动
  - ［无可乐］条件时，则转移到结束。







找饮料活动图示例（续）


找饮料
[无咖啡]
[无可乐]
[有咖啡]
[有可乐]
过滤器加咖啡
贮液器加水
取杯子
取可乐
过滤器放到机器上
开机器
调制咖啡
倒咖啡
喝饮料

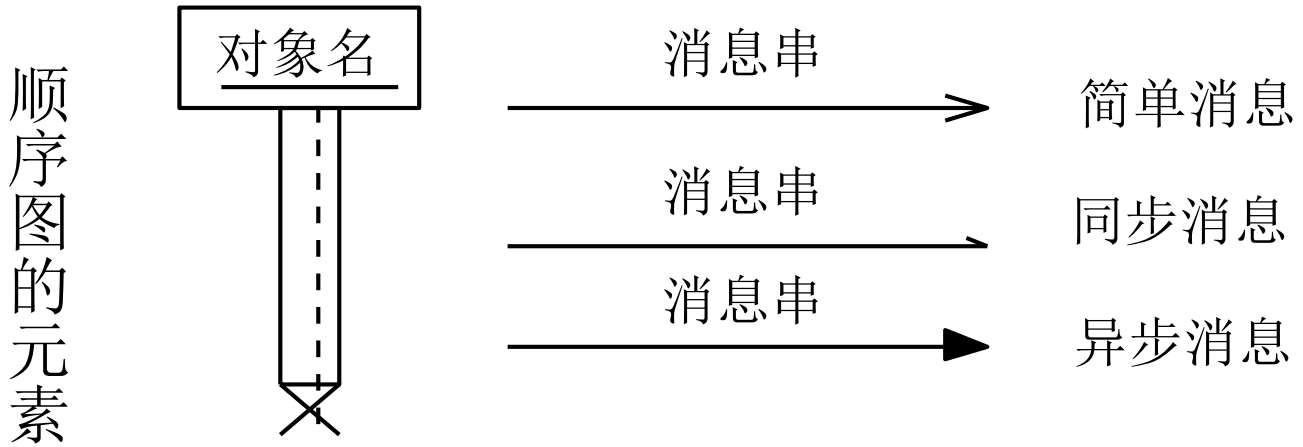
交互视图（interactive view）

- 对象行为是通过交互来实现的，交互是对象间为完成某一目标而进行的一系列消息交换。
- 消息（message）是对象间的单向通信，从发送者到接受者的携带信息的控制流。消息可能带有值参。
- 消息序列可用两种图表示：
  - 顺序图：重点在消息的时间顺序。
  - 协作图：重点在交换消息与对象间的关系。



顺序图（sequence diagram）

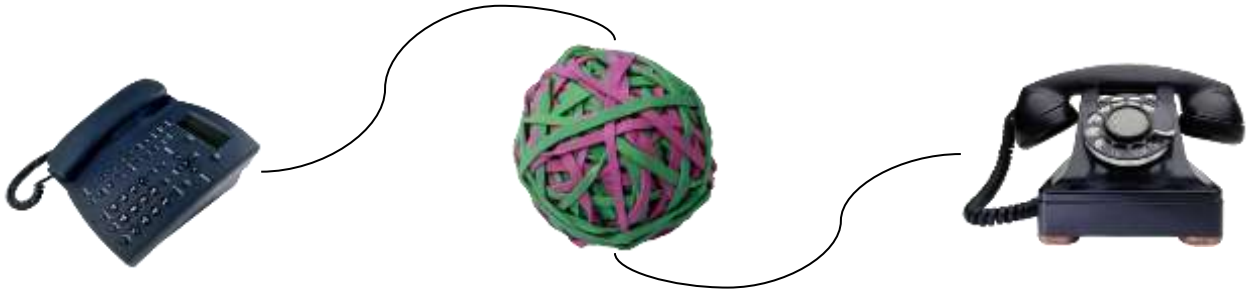
- 顺序图用二维表来表示交互，纵向是时间轴，横向是参与的角色以及它们交换的消息。
- 角色的生命周期表现为生命线，一条垂直的线，在激活的时间段里是双线，在状态保持的时间里是虚线。
- 消息表示为从一条生命线出发到另一条生命线的有向线，从上而下，表示消息的时间顺序。



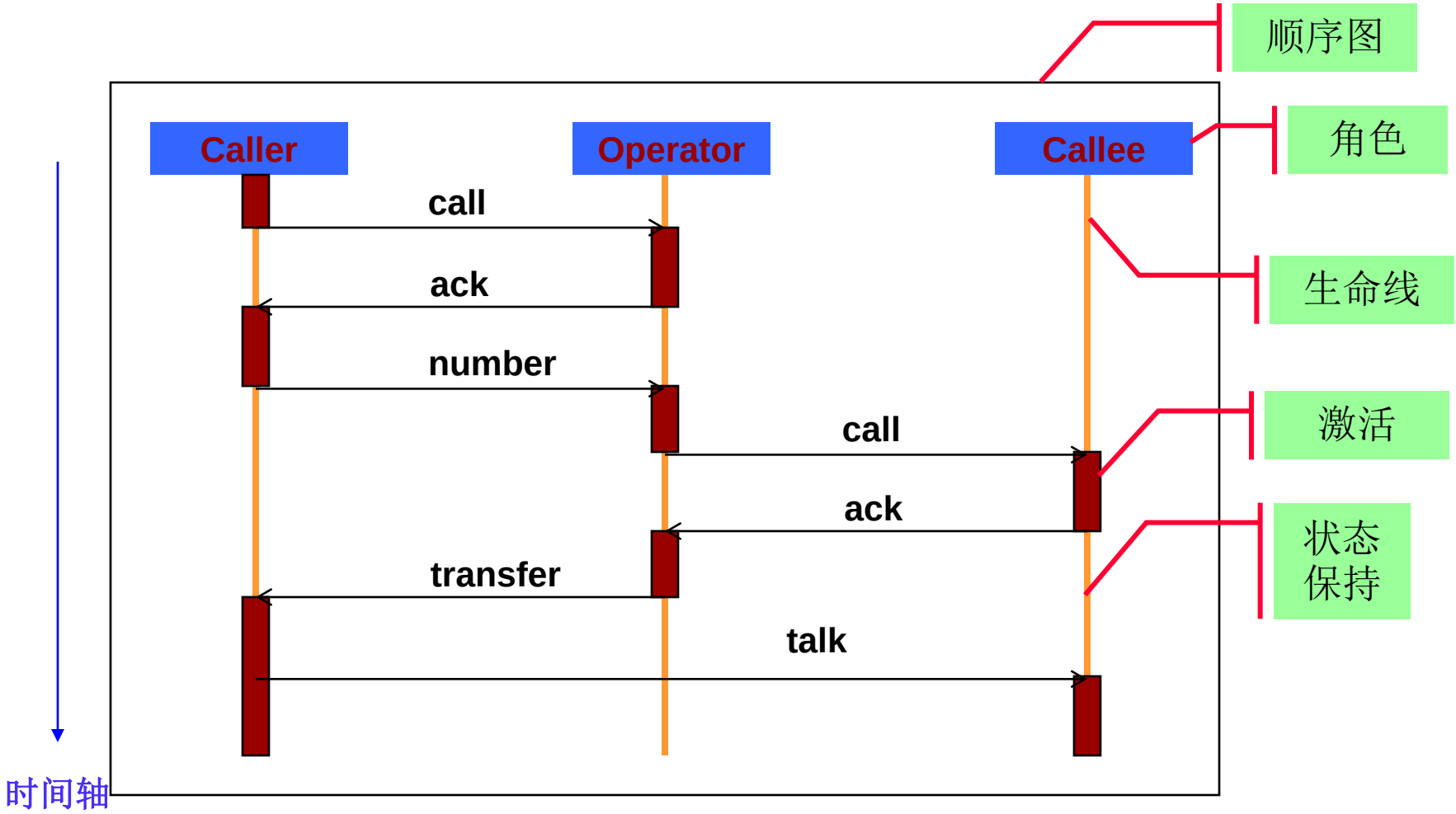
(a) 对象生命线　　(b) 消息

例：顺序图的示例

- 在一次电话通话中，涉及到的对象有：
  - Caller 呼叫者
  - operator 接线员
  - callee 被叫者
- 这些对象之间的消息发送和接受可以用顺序图描述。



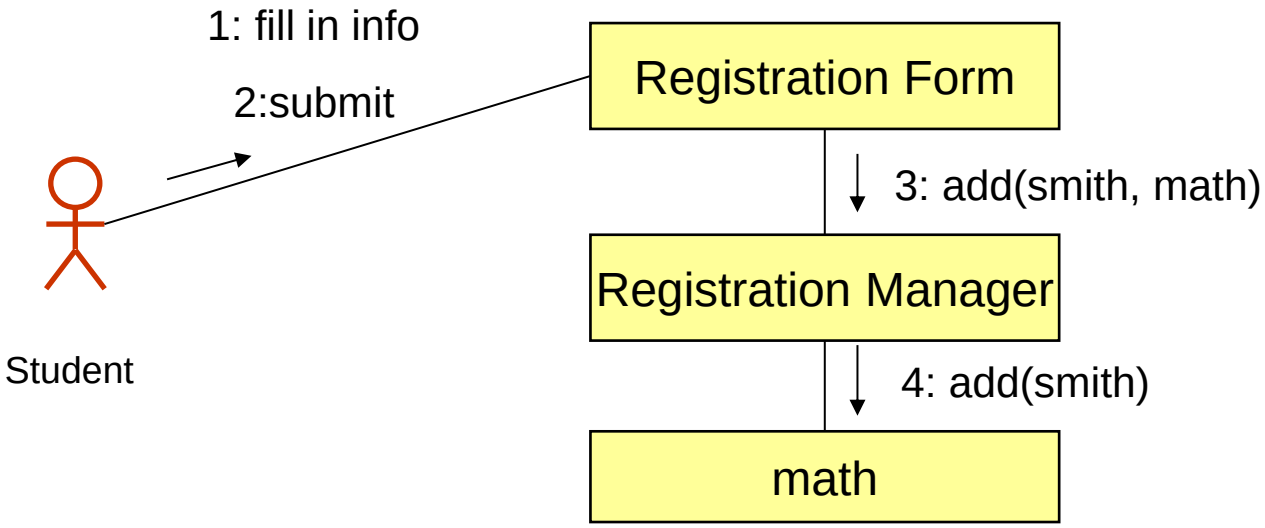
（续）顺序图


顺序图
角色
生命线
激活
状态
保持
Caller
Operator
Callee
call
ack
number
call
ack
transfer
talk
时间轴

协作图（collaboration diagram）

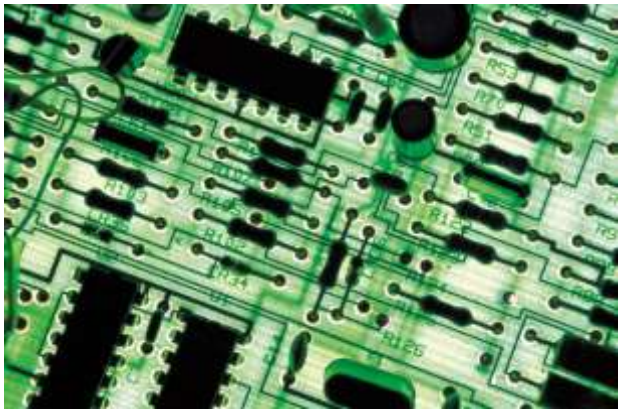
- 协作图显示的信息与顺序图相同，但显示方式不同，作用也不同。
- 协作图中对象的图示与顺序图中一样。
- 协作图着重于协作对象之间的交互和链接（消息），它可以表现系统中的操作、用例执行和或一个简单的交易场景。

协作图示例

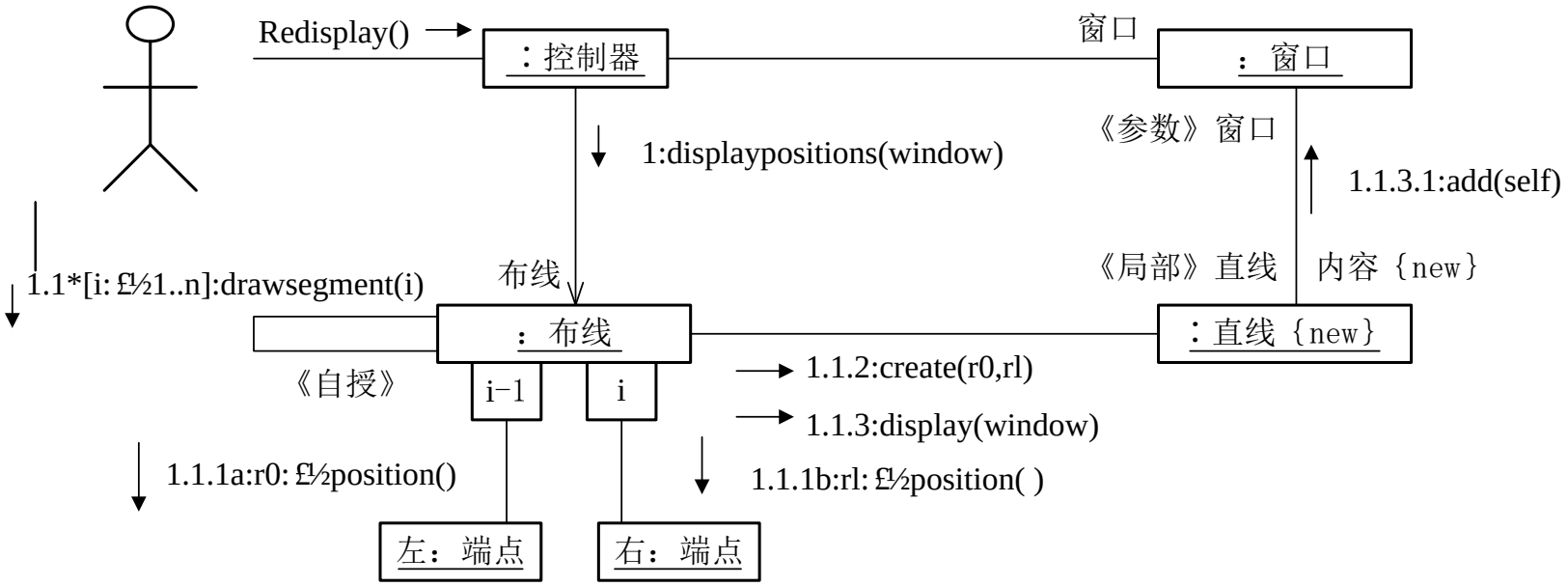


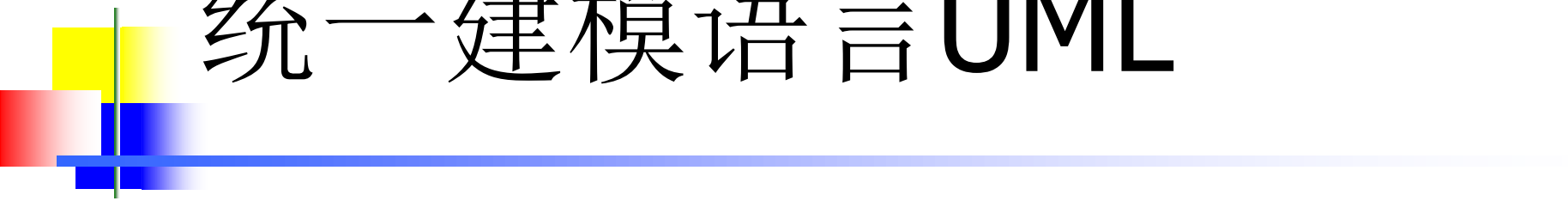
例：协作图的示例

- 在电路设计中，有“控制器”、“布线”、“端点”、“直线”及“窗口”等对象和类，在控制器控制下进行布线。
- 每布一条线， 先定位两个端点，即左端点r0、右端点r1，再以这两个端点为参数创建直线，并在窗口上显示出来，然后重复定位、 创建直线、显示，从而完成电路布线设计。



（续）协作图的示例


Redisplay()
：控制器
窗口
：窗口
1:displaypositions(window)
《参数》窗口
1.1.3.1:add(self)
1.1*[i:£½1..n]:drawsegment(i)
布线
《局部》直线
内容｛new｝
：布线
：直线｛new｝
《自授》
i-1
i
1.1.2:create(r0,rl)
1.1.3:display(window)
1.1.1a:r0:£½position()
1.1.1b:rl:£½position( )
左：端点
右：端点



统一建模语言UML

6 系统架构

系统架构（architecture）

- 系统架构(或称为体系结构)是对构成系统的各个部分的框架性描述。
- 在UML中，架构的定义是：架构是系统的组织结构。可以递归地把架构分解成：
  - 通过接口交互的部分
  - 连接各个部分的关系
  - 组装各个部分的约束。
- 系统架构分为两大类：
  - 逻辑架构（logic architecture）
  - 物理架构（physical architecture）



系统架构

逻辑架构

- 逻辑架构完整地描述系统的功能，把功能分配到系统的各个部分，详细说明它们是如何工作的。
- 在UML中，用于描述逻辑架构的图有：
  - 用例图
  - 类图
  - 对象图
  - 状态图
  - 活动图
  - 协作图
  - 顺序图

物理架构

- 物理架构详细地描述系统的软件和硬件，描述软、硬件的分解。
- 物理架构关心的是实现，因此可以用实现图建模，其中：
  - 构件图：显示代码本身的静态结构
  - 配置图：显示系统运行时的结构。

逻辑结构和物理结构应回答的问题：

逻辑结构：

① 系统提交哪些功能？

② 有哪些类，以及这些类之间如何相互联系？

③ 类和它们的对象如何协作以提交这些功能？

④ 系统的功能中有哪些时间约束？

⑤ 对哪些开发者来说，什么样的计划是合适的？

物理结构：

① 类和对象物理上位于哪个程序或进程？

② 程序和进程在哪台计算机上执行？

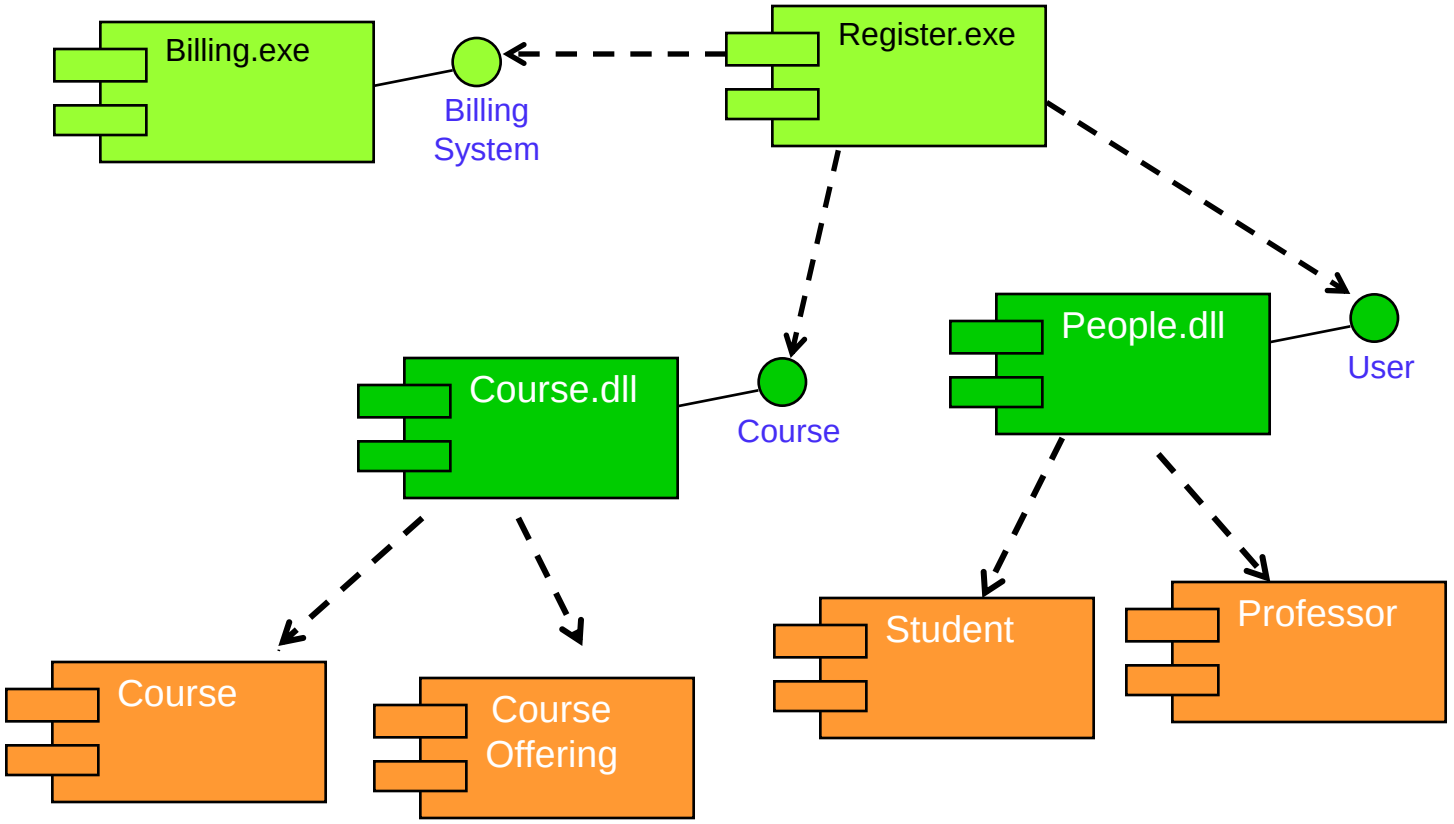
③ 系统中有哪些计算机和其它硬件设备？它们如何相互连接？

④ 不同的代码文件之间有什么依赖关系？如果一个指定的文件被改变，那么哪些其它文件要重新编译？

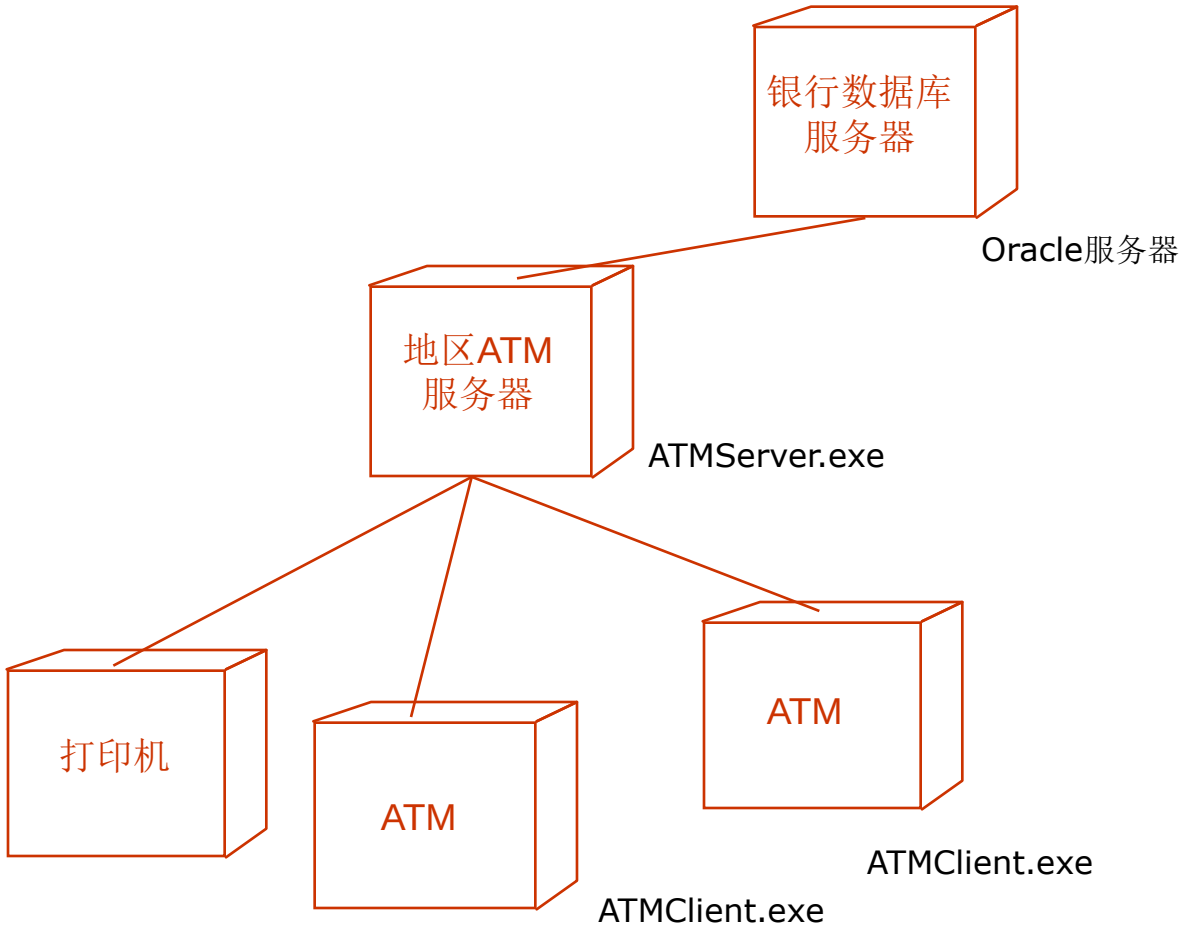
实现视图（implementation view）

- 实现视图描述可重用的系统组件以及组件之间的依赖关系。
- 实现视图一般的组成是：
  - 构件图（component diagram）
    - 描述构件及其之间的相互依赖关系。
    - 它通常是开发环境中的实现文件，包括：源代码构件、二进制构件和可执行构件。
  - 部署图（deployment diagram）
    - 描述处理器、设备和软件构件运行时的体系结构。
    - 描述系统资源在运行时的物理分布，系统资源成为结点。

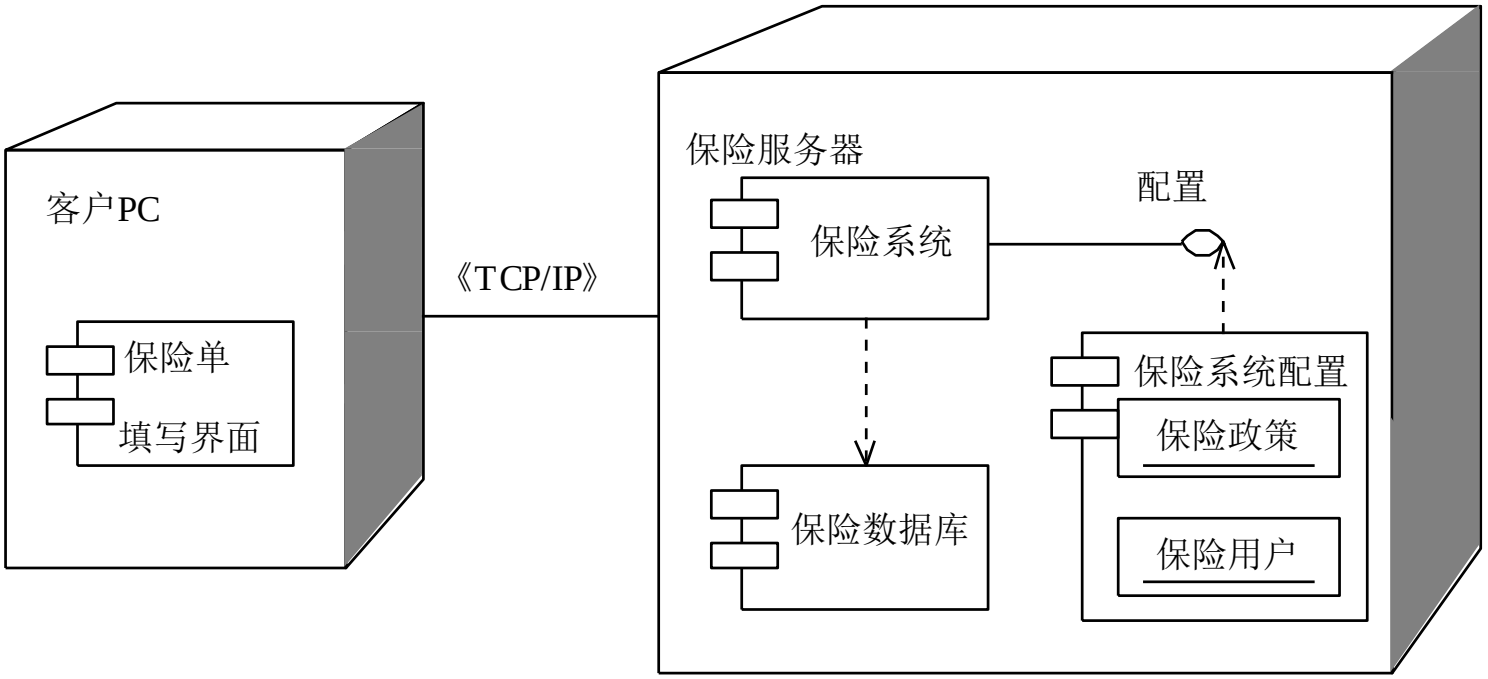
构件图（component diagram）



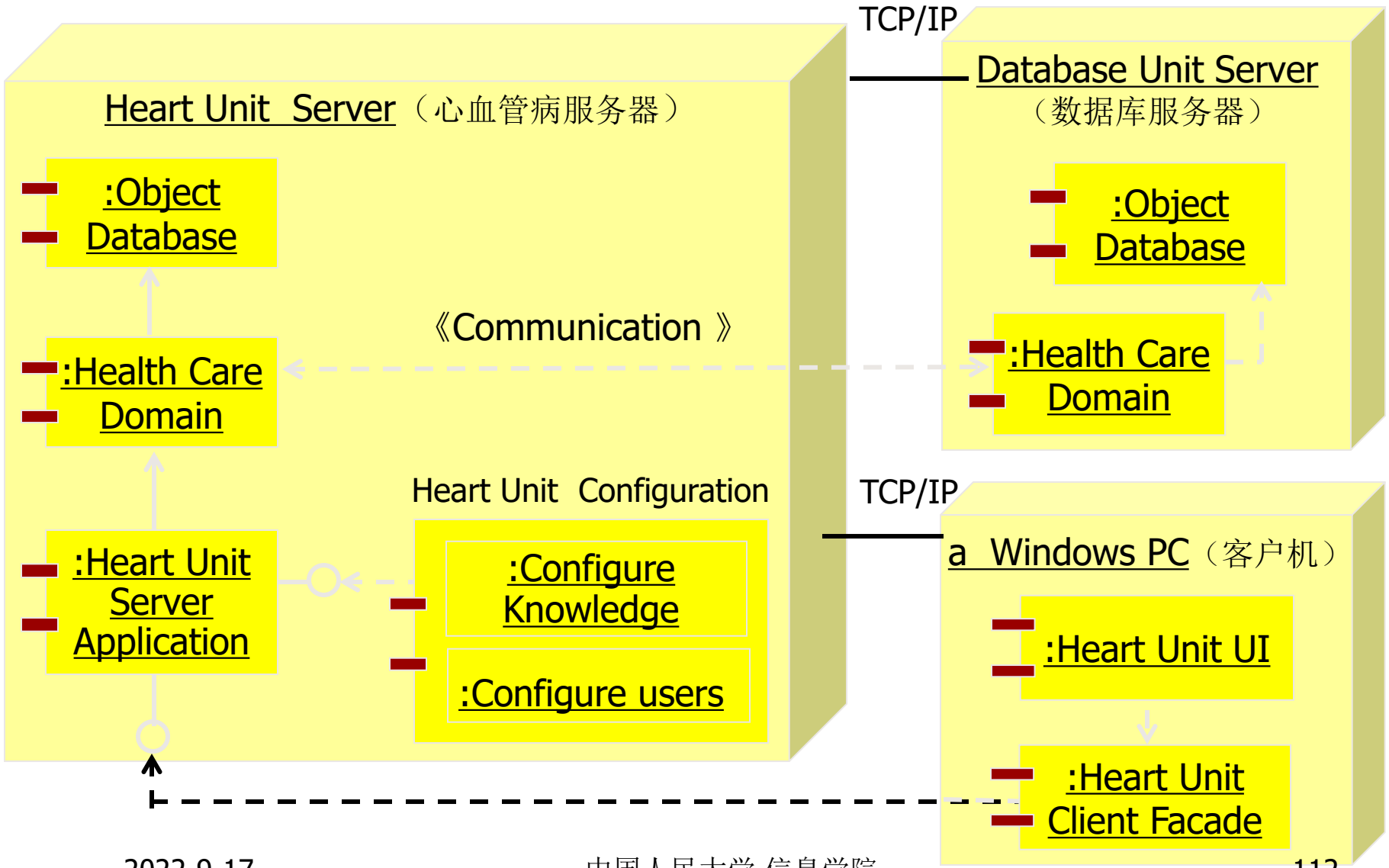
部署图（deployment diagram）

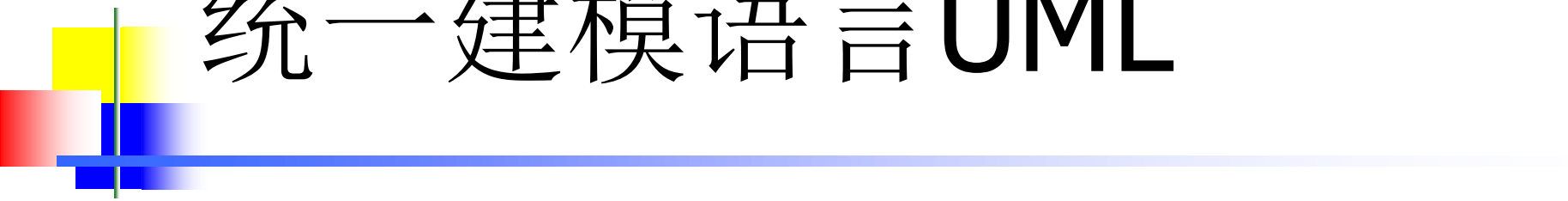


例：保险信息系统的配置图



例：医院诊疗系统的配置图





统一建模语言UML

7 建模步骤

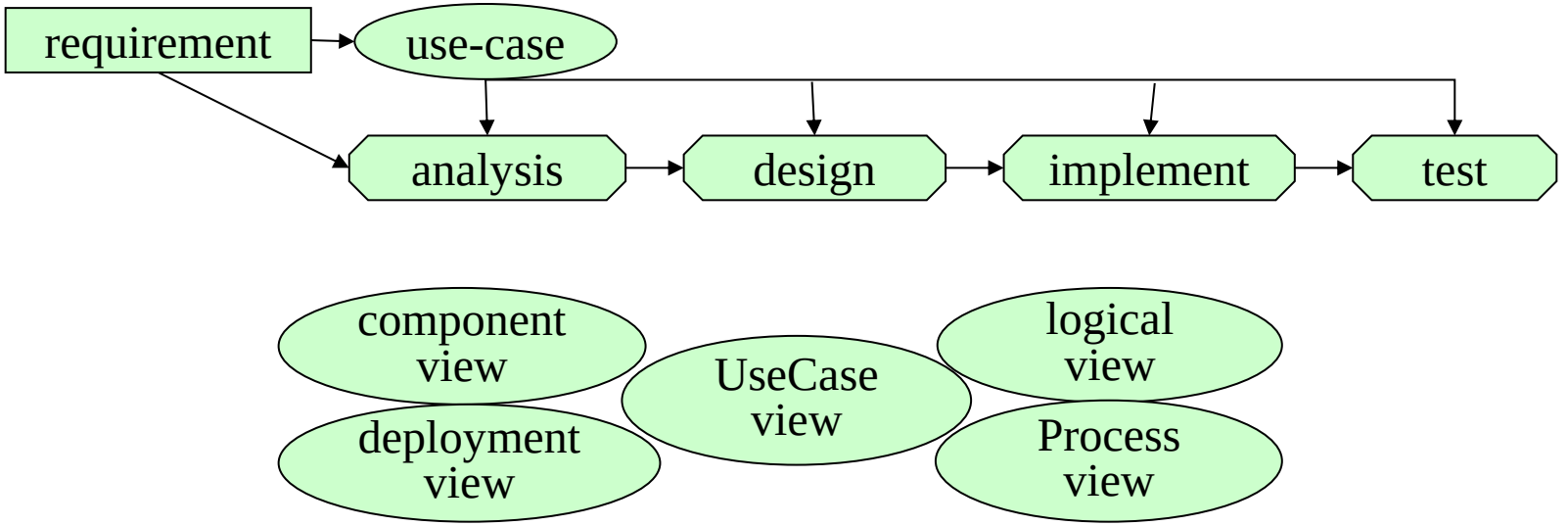
UML建模

- UML是一种建模语言而不是建模方法，这是因为它没有过程的概念，而过程正是方法的一个重要组成部分。
- UML本身独立于过程，这意味着用户在使用它进行建模时，可以选用任何适合的过程。
- UML过程的基本特征：
  ① 用例驱动（use-case-driven）
  ② 以体系结构为中心（architecture-centric）
  ③ 迭代（iterative）
  ④ 增量（incremental）



用例驱动的系统

- 用例获取系统的功能需求，它们驱动需求分析之后的所有阶段的开发。
- 用例包含了功能描述，所以它影响所有阶段的视图。



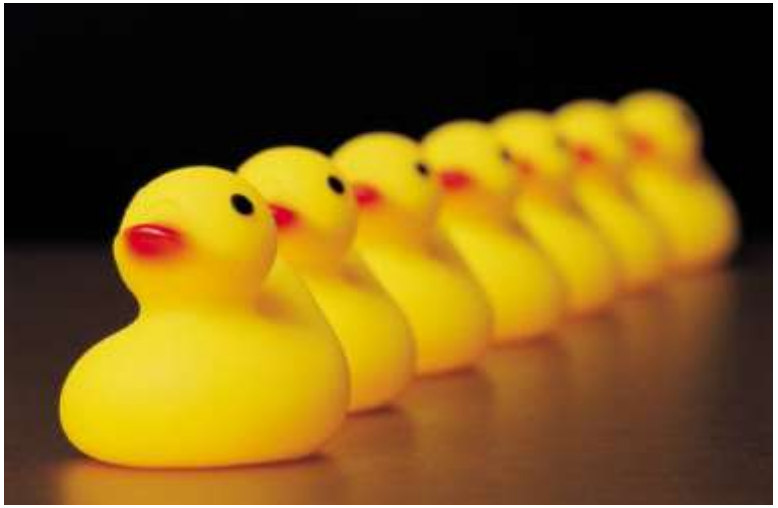
以体系结构为中心

- 在项目早期定义一个基础的体系结构非常重要，然后将它原型化并加以评估，最后进行精化。
- 一个好的体系结构涉及到功能和非功能两个方面，尤其重要的是定义一个易修改、易理解和允许复用的系统。
- 创建体系的一个重要工作是逻辑上把系统划分成子系统（UML中的包）。



迭代

- 用UML建模不要试图一次定义模型或图的所细节，每次迭代都要增加一些新的信息和细节。
- 每次迭代都要评估，并用于下一次的输入。评估考虑这次迭代对系统的最大影响或最高风险。



增量式

- 每次迭代都包含分析、设计、实现、测试等阶段，通过若干次渐进式增量开发完成整个系统的开发。
- 增量式开发过程分解了开发的风险，不至于把风险留到开发的后期。



增量迭代模型的UML建模过程

① 需求

② 分析

③ 设计

- 结构设计
- 详细设计

④ 实现

⑤ 测试和配置



（1）需求

- 最初的需求规格说明应当由代表系统最终用户的人员提供
- 需求的内容包括系统基本功能需求和对计算机系统的要求。

（2）分析

- 分析的任务是找出系统的所有需求并加以描述，同时建立模型，以定义系统中的关键领域类，应由系统用户和开发人员合作完成。
- 分析的第一步是定义用例，以描述所开发系统的外部功能需求。用例分析包括阅读和分析需求说明，此时需要与系统的潜在用户进行讨论。

（3）设计

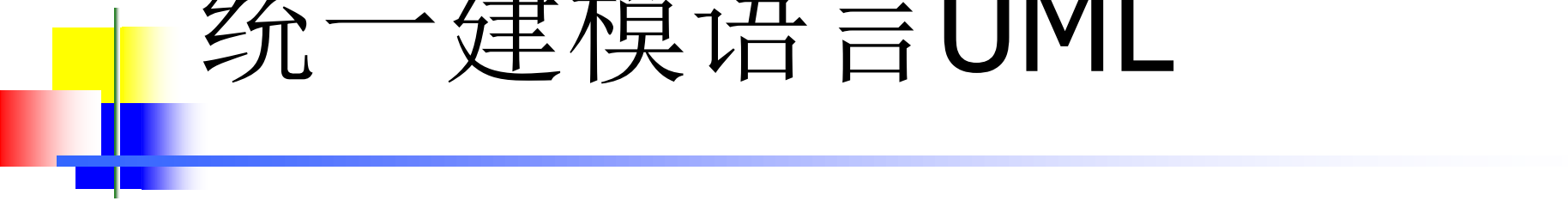
- 设计阶段的任务是通过综合考虑所有的技术限制，以扩展和细化分析阶段的模型。
- 设计阶段可以分为两个部分：
  ① 体系结构设计是高层设计，其任务是定义包（子系统），包括包间的依赖性和主要通信机制。
    - 一个设计良好的系统结构是系统可扩充和可变更的基础。
    - 包实际上是一些类的集合。
    - 类图有助于用户从技术逻辑中分离出应用逻辑（领域类），从而减少它们之间的依赖性。
  ② 详细设计，细化包的内容，使编程人员得到所有类的一个足够清晰的描述。
    - 详细设计的目的是通过创建新的类图、状态图和动态图（顺序图、协作图和活动图），描述新的技术类，并扩展和细化分析阶段的对象类。

（4）实现

- 构造或实现阶段是对类进行编程的过程。
- 可以选择某种面向对象编程语言（如：Java、C++）作为实现系统的软件环境。Java很容易实现从逻辑视图到代码部件的映射，因为类到Java代码文件之间是一一映射关系。
- 在实现阶段中，可以选取各种图的说明来辅助编程，比如：类图，状态图和动态图等。

（5）测试和配置

- 完成系统编码后，需要对系统进行测试，它通常包括：
  - 单元测试：在单元测试中使用类图和类的规格说明，对单独的类或一组类进行测试；
  - 集成测试：在集成测试中，使用组件图和协作图，对各组件的合作情况进行测试；
  - 系统测试：在系统测试中，使用用例图，以检验所开发的系统是否满足例图所描述的需求。
  - 验收测试：
- 系统的配置是实际地交付系统，包括文档和组成模型等。



统一建模语言UML

8 UML总结

UML总结

- UML（Unified Modeling Language）是一种构建软件系统和文档的通用可视化建模语言。
- UML能与所有的开发方法一同使用，可用于软件开发的整个生命周期。
- UML能表达系统的静态结构和动态信息，并能管理复杂的系统模型，便于软件团队之间的合作开发。
- UML不是编程语言，但支持UML语言的工具可以提供从UML到各种编程语言的代码生成，也可以提供从现有程序逆向构建UML模型。

UML几种图的区别

- 模型图各有侧重
  - 用例图侧重描述用户需求，
  - 类图侧重描述系统具体实现；
- 描述的方面都不相同
  - 类图描述的是系统的结构，
  - 序列图描述的是系统的行为；
- 抽象的层次也不同
  - 构件图描述系统的模块结构，抽象层次较高，
  - 类图是描述具体模块的结构，抽象层次一般，
  - 对象图描述了具体的模块实现，抽象层次较低。

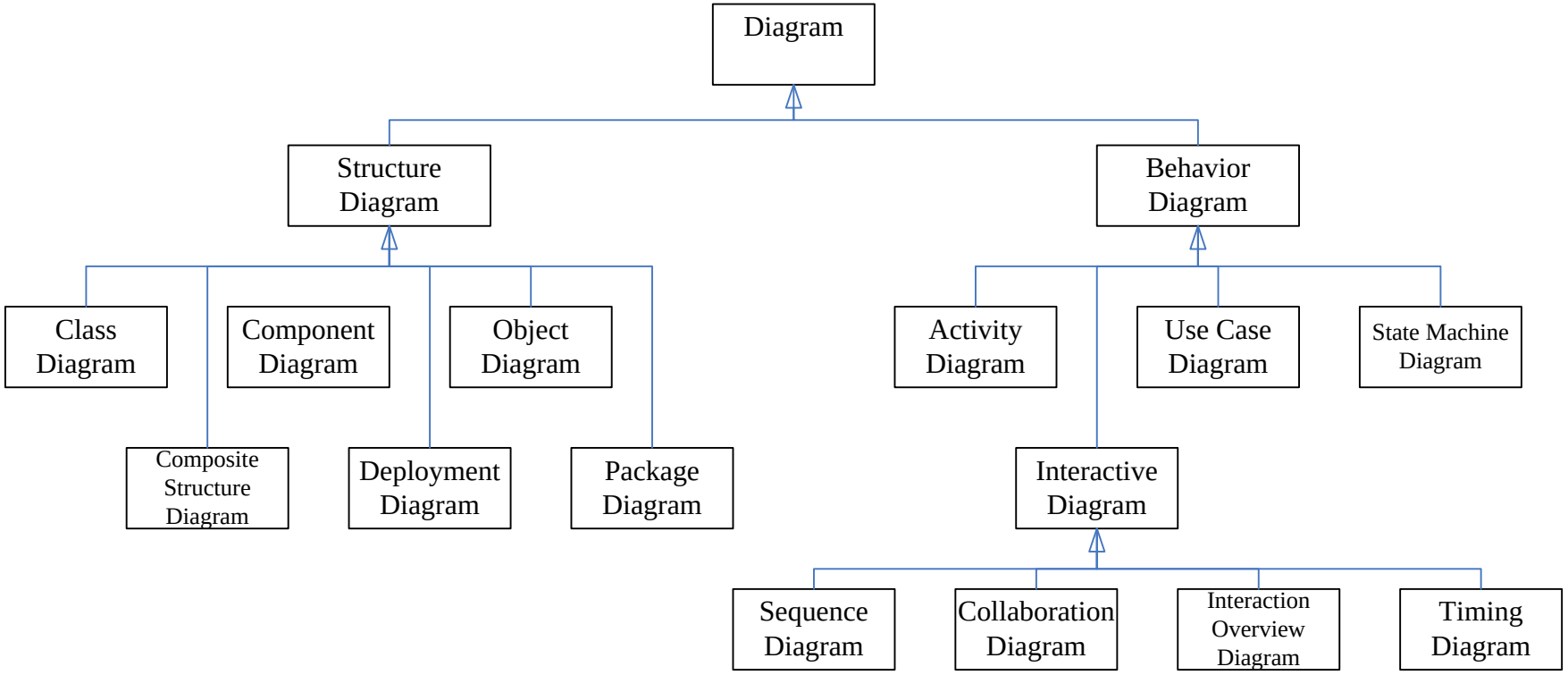
使用UML的准则

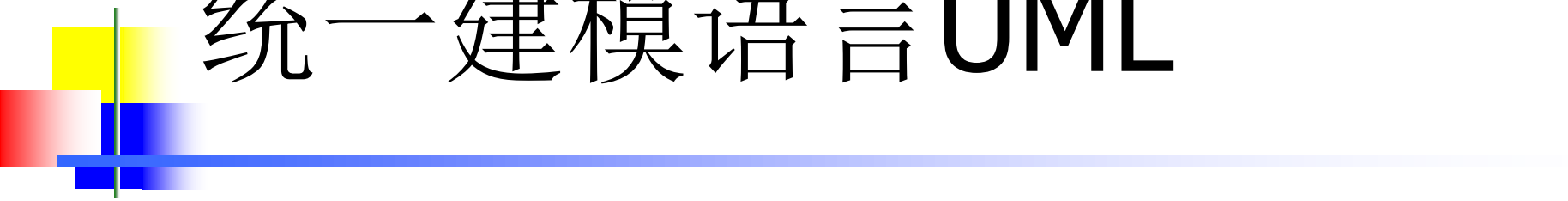
- 不要试图使用所有的图形和符号
  - 应该根据项目的特点，选用最适用的图形和符号。
  - 一般来说，应该优先选用简单的图形和符号，例如，用例、类、关联、属性和继承等概念是最常用的。
- 不要为每个事物都画一个模型
  - 应该把精力集中于关键的领域。最好只画几张关键的图，经常使用并不断更新、修改这几张图。
- 应该分层次地画模型图
  - 根据项目进展的不同阶段，用正确的观点画模型图。
  - 如果处于分析阶段，应该画概念层模型图。
  - 当着手软件设计时，应该画说明层模型图。
  - 当考察某个特定的实现方案时，则应画实现层模型图。
  - 使用UML的最大危险是过早地陷入实现细节。为了避免这一危险，应该把重点放在概念层和说明层。

（续）

- 模型应该具有协调性
  - 模型必须在每个抽象层次内和不同的抽象层次之间协调。
- 模型和模型元素的大小应该适中
  - 过于复杂的模型和模型元素难于理解也难于使用，这样的模型和模型元素很难生存下去。
  - 如果要建模的问题相当复杂，则可以把该问题分解成若干个子问题，分别为每个子问题建模，每个子模型构成原模型中的一个包，以降低建模的难度和模型的复杂性。

UML 2.0中图的分类


Diagram
Structure Diagram
Behavior Diagram
Class Diagram
Component Diagram
Object Diagram
Activity Diagram
Use Case Diagram
State Machine Diagram
Composite Structure Diagram
Deployment Diagram
Package Diagram
Interactive Diagram
Sequence Diagram
Collaboration Diagram
Interaction Overview Diagram
Timing Diagram



统一建模语言UML

9 UML建模工具

建模工具

- IBM Rational Rose
- Microsoft Visio Professional
- PowerDesigner
- Select Component Architect
- Visual UML
- Borland Together
- Gentleware Poseidon
- astah
- OmniGraffle

课堂讨论：

(1) 识别UML的图形符号，叙述每种UML图的作用。

(2) 对下列领域之一进行概略的领域分析，并用UML的图形符号来表示分析的成果：

a) 大学生成绩记录系统

b) 大学生选课系统

c) 银行的客户服务

d) 手机短信服务

课堂练习题

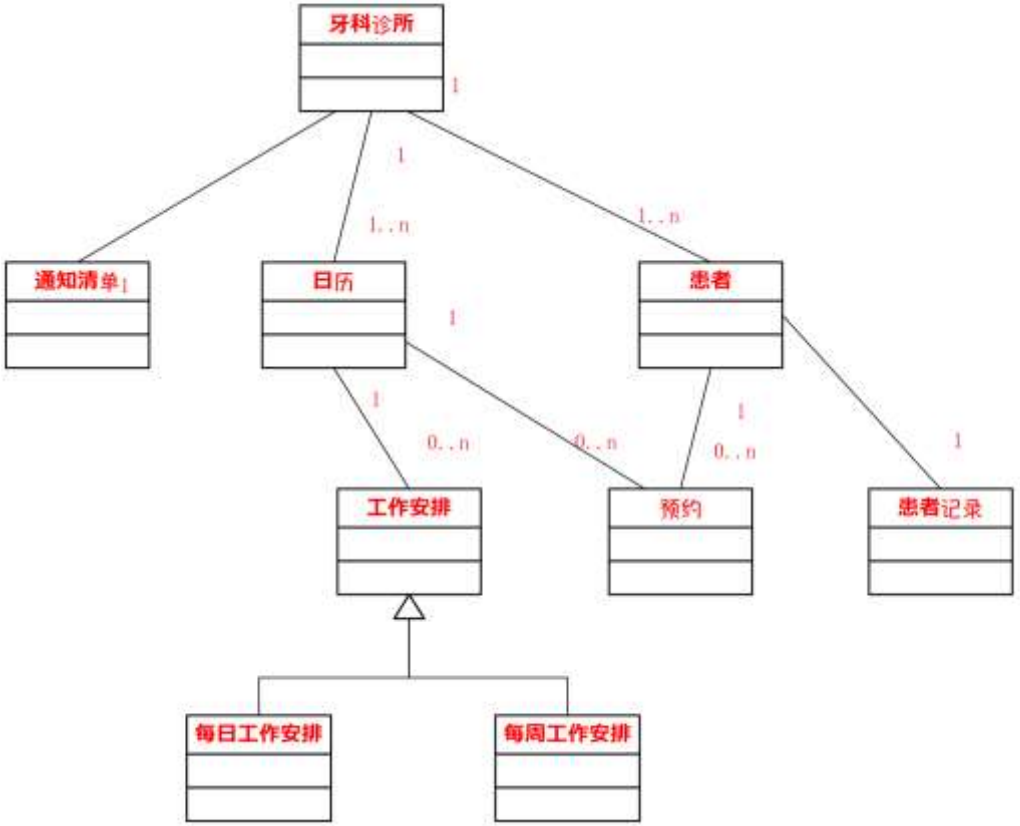
确定牙科诊所问题陈述的对象，用UML描述问题静态模型：

汤姆在小镇上开了一个牙科诊所。他有一个牙科助手、一个牙科保健员和一个接待员。汤姆需要管理预约系统。

在患者打电话预约时，接待员将查阅日历，并安排患者尽早诊治。如果患者对约定时间感到满意，接待员将输入约定时间和患者名。系统将核实患者名并提供患者的记录数据，这些记录数据包括患者ID号等。在每次检查或洗牙后，保健员或助手将标记相应预约已完成，增加说明，如果必要，会安排患者下一次再来。

系统要响应按患者名和按日期进行的查询。患者记录数据与预约信息一起显示。接待员可取消预约，可打印出前两天预约尚未接诊的通知清单。系统中有来自患者记录的电话号码。接待员还可打印出有关所有患者每天和每周的工作安排。

静态模型


牙科诊所
1
1
1..n
1..n
通知清单
日历
患者
1
1
1
0..n
0..n
1
0..n
1
工作安排
预约
患者记录
每日工作安排
每周工作安排

习题

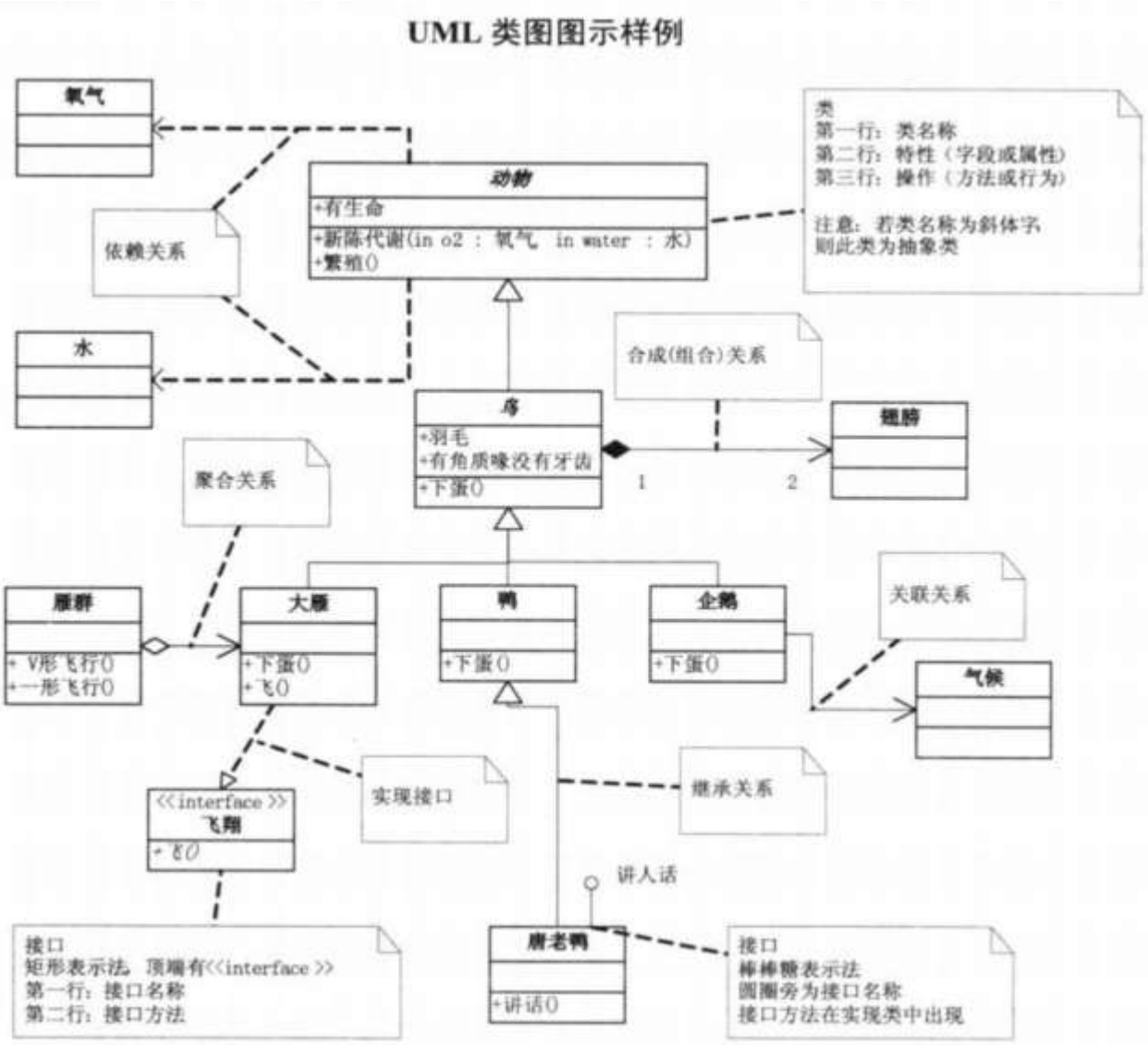
(1) 什么是OOA建模语言，它应该包括哪些方面？
(2) 什么是Use-Cases模型，主要作用是什么？
(3) UML中有哪些动态建模的工具，各表达什么内容？
(4) UML中的接口类有什么特点，举例说明。

练习：读类图

① 解释类图表达的对象及对象之间的关系。

② 重点说明的类：
- 水
- 气候
- 飞翔
- 翅膀

③ 重点说明的关系：
- 动物←→大雁
- 雁群←→大雁
- 唐老鸭←→鸭



练习题：图书馆管理系统建模

- 图书馆管理系统中，读者包括教师、学生（学生中有本科生、研究生、）等等，图书包括杂志、各类书籍。
- 图书管理员：图书馆工作人员，主要有以下权限：分类管理、借书处理、还书处理、解除预定
- 图书借阅者：是系统中最多也是最重要的参与者，主要有以下权限：查询个人信息、查询图书信息、预定图书、借阅图书、返还图书。
- 要求：

1. 请用UML建立UseCase图
2. 读者和图书的类图
3. 图书的状态图